Welcome to the module, Databases and Advanced Data Techniques.
Play video starting at ::17 and follow transcript0:17
In this module, we're going to be looking at data. And we're going to be looking at it practically from a few sort of activities, from the angle of a few different activities. And those will be, we'll be understanding the structure of the data and modeling it. We'll be getting data and cleaning it for use. Well be looking at how to store and retrieve it, particularly within databases. And we'll be looking at using and sharing it. And these are very abstract, very high level ideas will be looking fairly practically. But of course, we can only go so deep. And that's a refrain that will be coming back in different forms throughout this module. Because if I want to model something in the real world through represented digital data, if I wanted to model every single aspect of it and all of its nature. No matter how small it is, whether it's a teapot, or a sofa, or a computer, or the readings from the large Hadron collider, I would have too much data and too much complexity of representation. So representing real life objects and working with them in databases is necessarily selective process, a distorting process, and something that gets its nature from the person involved. So it's a function of both the real world object that we're modeling and the modeler on what their preferences and biases are.
Play video starting at :1:50 and follow transcript1:50
That's also true of this course, it comes as a result of the selections, the models and the point of view of the person who designed it. So I should probably at this point introduce myself. My name is David Lewis and my research special is digital musicology. Digital musicology is a branch of digital humanities and that will affect the sorts of approaches I take in various different ways.
Play video starting at :2:21 and follow transcript2:21
Digital humanities research is generally a bit smaller than say, computational astronomy, bioinformatics, or other data heavy science tasks. It's also a little bit more complex in terms of the data that it represents. It's also a bit more complex and nuanced than say, in the commercial world, in e-commerce, the databases that you might use for that. Our data is often smaller, more contingent, more historical, possibly more ambiguous, possibly more likely to be wrong. And it's also a bit more varied. So on the one hand, we'll have information that's historical. We might have social information, but we might also have recordings of music and we might also have sensor readings from people who were listening to some music where we're measuring heart rate, skin conductivity, and so on.
Play video starting at :3:29 and follow transcript3:29
We might also have music notation, we might have video. So we deal with quite a diverse range of data types, but often quite small compared to other field's volumes of data. And that will color I'm sure the sorts of things that we talked about in the module ahead. In the first topic that we're going to be going into, we're going to be starting just from that point of view, talking about what your data might be, how you need to look at it, and what shapes it might take.

Over the course of this module, we are going to be talking about how to structure data, we're going to be talking about optimizing our databases, building and sharing database applications. But before we start talking about that, perhaps we should first address the question of where our data is going to come from in the first place. Where does your data come from? Well, it's difficult

to generalize, but we can put things into a couple of broad categories. It may be that you'll be making your data new, specially, for this case, your build it either with or after the database application itself. Or it may be that there is data that's preexistent. In that case, it may be that that data is data that you or your organization already has form legacy data. Perhaps the application you're building is actually a continuation or an optimization of something that previously existed and that you are taking it to a higher level of sophistication. Perhaps your organization used spreadsheets for organizing its data before and now wants to move to a database. The alternative source for data will be external sources, people, organizations who have data that you can use. Either data you find or data you buy in or is provided to you by suppliers. Let's take each of those at a time because each will have its own processes and its own problems. New data is perhaps easiest when you build it with the database application, adding data as you go. An example of that might be an organization that has appointments, say a hairdresser's has bookings with clients and when a new system comes online, any new bookings they take get added to the database. The database gets bigger, slowly, and can be tested effectively on a small dataset, and the older bookings will stay in the old system and that way, you never have a problem with the older data in the first place. In many other cases though, you won't be able to accommodate a half system. You'll have to have all your data at once, and in those cases, you will need to engage some way of getting bulk data into your system. You may have to pay someone to put in that data. Or if you're a research organization or a charity, you might get the generosity of the public in the form of some crowd-sourcingggggggggggggg, and that's something we'll look at briefly in some of the readings later in this topic. If you've got preexisting data, then there are some processes you would expect to have to go through, to get it in the form you needed. You might well need to extract it if it was already in a database or in spreadsheet, then you'll need to get that data out in a form that's useful to you. You may need to convert that data to a form that is useful or that satisfies your new requirements, and then maybe what's good at cleaning stage. A stage of checking the quality and the level of detail of your data so that it's appropriate for your new database.

Play video starting at :4:1 and follow transcript4:01

Why might you want to go to an external source rather than doing everything in house? Well, for a start, you don't have to pay for data entry. You don't have to pay for quality checks, and that's not just a financial question. It can be in terms of manpower and managing the process, in the expertise needed to manage a data entry process. You're not just delegating the process of entering data, but you may also be delegating expertise. If an organization has particular knowledge about a product or a domain, then getting them to enter the data means taking advantage of their specialisms. It's rare for an organization to be excellent at everything. So getting data from multiple sources can be a way of reducing your need for internal expertise. Of course, the flip side of having all that freedom, losing all that, delegating all that power is that you now have much less power. You have little or no control over the data quality and the data structure would have been decided by the individual or organization that is providing the data based on either their own needs or their perception of their clients needs. Your needs may be different. You may require different data structures. Data may be incomplete from your point of view. What your interested in and what they're interested in may be different. It may be ambiguous from your point of view which is to say that the level of detail at which a particular

piece of information is encoded is different to the level of detail that you need. Of course, there are questions of how much you trust the data that is being provided by this third party. Whenever you're bringing data into a database, into your application, you have to think about a set of issues. Evaluate the pros and cons of getting your data from any particular source, and we'll look a little bit at how to do that in this topic.

When we looked at the idea of getting data from external sources, I made the point that
Play video starting at ::17 and follow transcript0:17
sometimes the external source of information may seem to be ambiguous or incomplete, because the expectations of what data was needed, how much detail was needed, and how it would be used will be different amongst different users, different groups. So when you come to model
Play video starting at ::41 and follow transcript0:41
real life data and prepare to put it into a database, you'll have to be thinking about what the use of that data is, what sort of information you will be interested in. Let's take a simple example. What is the data that we might want to encode in a database for
Play video starting at :1:7 and follow transcript1:07
this object? Well, it's a book. Maybe we should start there. If I were an online researcher
Play video starting at :1:19 and follow transcript1:19
and I was thinking of posting this to a customer, then I might care particularly about its weight.
Play video starting at :1:29 and follow transcript1:29
So let's have that in the database. It's a little over 500 grams, and maybe I'd care about its other dimensions. If I were a librarian and storing this in storage, I might care
Play video starting at :1:43 and follow transcript1:43
about the shelf height that it needs. So let's put height into the database. What other information
Play video starting at :1:52 and follow transcript1:52
do we have? Well, it's red and it's green, and it's got a red spine. If I were trying to use this to make my room's bookshelves look pretty, I might care about that. What other information might be useful? Well, I'm sure you'll already have ideas about what
Play video starting at :2:15 and follow transcript2:15
sort of information you'd expect in a database about books. You might expect a title. One
Play video starting at :2:25 and follow transcript2:25
use for that is that if I had this as a librarian or a bookstore, and I had this on a shelf, I might need the title to find it, as in I would look along the shelf and, there we go, be looking for The Gardener's Calendar. On the other hand, if I wanted to compare this with other examples of this book, the title might come from the title page. But here we see something slightly different. Here we see Every Man His Own Gardener, being a new and much more complete gardener's calendar and general directory than any one hitherto published. Now that does contain gardener's calendar in that longer title, but it's a longer, more extensive title. So we might need that for comparison with other editions.
Play video starting at :3:15 and follow transcript3:15
We should also bear in mind that at the beginning of the book text itself, it presents us with a third form of the title, Every Man His Own Gardener. So we not only have to think about
Play video starting at :3:30 and follow transcript3:30

the use we want to put it to when we decide which categories of information we want to store, whether it's weight or height or title, but also when we're interpreting that information, what we're going to store. For the point of view of finding this book on a shelf, all we want is gardener's calendar. From the point of view of finding other examples of this book or books with similar titles, we might want the long form or Every Man His Own Gardener.
Play video starting at :4:2 and follow transcript4:02
What other information might we care about? Well, we might care about authors, and the authors of this book are Thomas More and, it says on the title page, Thomas More and John Abercrombie, and we even have a picture of John Abercrombie. Now Thomas More, the first named author, the gardener, to his grace, the Duke of Leeds, only really has his name on here because he was a famous gardener to a famous man, and it would help sell the book. If we cared about the intellectual origins of this book, we might want to indicate who was the intellectual contributor, the man who wrote the book itself, but for most retrieval purposes, we're probably more concerned about having both names. Date would be useful. It might be interesting for historians. It would also be interesting for antiquarians trying to sell the book. This book was published in 1803, and it says so on the title page, but it's also the 17th edition, so much of the content will have come from decades earlier. So, have we covered all the information anyone might want from this book? Well, to some extent we've hardly started. It's a book, it's full of information, and we haven't touched that yet. If I were an historian of gardening, I would really care about what this particular author thinks about pruning fruit trees in the late 18th century, and only if I encode the content in a way that allows me to find that, can I begin to study that using the database tools. So, even in a simple item like a book, you can see that different interests will dictate quite strongly the shape and the content of the data that we want to represent.

If someone is going to let me use their data or I'm going to let them use mine, there are some things we're going to have to agree on. For a start, we're going to have to talk about what we mean by my data, their data, your data. We're also going to have to know who we can pass that data onto beyond ourselves, how widely we can distribute it, and under what terms? We're also going to have to consider what we're allowed to do with that data. Whether there are any restrictions that control what we can do, how we can process it. In academic and government circles, it's quite common for there to be a philosophy of making data as openly available and as unrestricted as possible. But that's by no means universally even within government and it's certainly not universal when we get to the commercial world. These restrictions are essentially legal in nature or they're applied through legal channels. You may have met some of them in both data terms, but also particularly in terms of software licensing and the same challenges apply to some extent that we have data that transcends national boundaries that we might make available on the web and so have very little control who, in what territory he is accessing the data. But on the other hand, it is data that's governed by local national rules both in the place where the data's being distributed from and in the places where it's being consumed.
Play video starting at :2:7 and follow transcript2:07
This diagram represents all the sources of linked open data as defined by Lod-Cloud in January 2021. Every circle represents the data source that is freely available in a particular form that's been published on the semantic web and shared. Each circle is a resource, each line is use of

that resource by another results, so a co-dependency or dependency. This is a big graph, there is a lot of data embodied in this graph. But this is a small proportion of the range of ways in which data is made available for free.
Play video starting at :3:3 and follow transcript3:03
The question we can ask is why is so much information being shared for free if information is so valuable?
Play video starting at :3:20 and follow transcript3:20
The catalog of a furniture company. In here is details of every product that they make, the sizes, the materials, the prices, product codes, photos showing them in different contexts. If I use the app, I can see the instruction manuals for putting this together, which will list every single part and every single nut and bolt that's needed to make these items of furniture. In principle, I could copy the furniture from them. There's lots of valuable data here, why is it being given away for free?
Play video starting at :4:6 and follow transcript4:06
Well, I mean, if I didn't know that a product existed, I couldn't go and buy it. A company that sells a large, diverse range of products requires us to know about them. We need to find out about them before we can buy them. So sharing catalog data, sharing information about your products, is vital for selling them. That's not just for physical products, music publishers will share detailed information about the artists and dates and so on for their releases for their tracks with streaming companies. Because the streaming companies will need to sell them and so they need to be discoverable. To aid discovery, you need to make data available. There's a strong commercial reason why you might make your data available free and as widely as possible. When we come to ask why you would publish open data, we've already come up with one answer. Commercial reasons. You might want to use that data to drive sales of a product.
Play video starting at :5:24 and follow transcript5:24
You may have ethical reasons. You may want to support the common good. You may feel that by sharing your data, you make the world a better place. Researchers may want to share their findings as widely as possible because they think it's valuable and because they think that people should know. Similarly, enthusiasts will often want to share their passions as widely as possible. They want other people to think that the artists they love or the activity they love is as exciting as they do themselves, and they will often share vast amounts of data to help with that process. There may, on the other hand, be practical contractual requirements that force you to make your data open. That's particularly the case with government and with research organizations. For example, the United States government has an expectation that things that are paid for by the public, by the tax payer should be owned by the public and therefore available to them and that assumption gets baked into a lot of contracts that require that data to be openly available to the public. Similarly, a lot of research founders feel that for the public good, researchers should make their data available freely and as a result, research funding contracts may have a requirement for that baked into them.
Play video starting at :7: and follow transcript7:00
Another more commercial reason might be that each of several organizations produces a little bit of a large picture, and that the only way to make an application that works is for them all to pool their data, to share to make something bigger, to use interoperability to create some [inaudible]. Those are very strong reasons why you might publish data openly. Why might you

not? Why wouldn't you make everything available to everyone? There may be restrictions on the data that forms a part of your data set itself. It may be that something about what you've been collecting can't be shared. Some of that may be because it's commercially sensitive or there's some intellectual property attached to it. Some of it may be for privacy reasons. If you're a medical researcher, you may have personal information about the medical records of individual people. In those cases, you may not be able to make that data available without making some small changes or large changes to the data set.
Play video starting at :8:21 and follow transcript8:21
There may be a wish to control how your data's use. If it's artistic data or of its commercial data, you might want to restrict who sees or who uses it and how it's used.
Play video starting at :8:36 and follow transcript8:36
The data itself may be the product. If you're a furniture company, then furniture is your product, and information about it is something that drives sales of the product. On the other hand, if you've spent a lot of money gathering data and want to recoup that money then yourself will want to charge for it, you wouldn't make it openly available. A simple example, the post office in the United Kingdom has what's called postcode, that's a six or seven character code that identifies quite tight geographical area. Just a house number plus a postcode gives you a distinct location in the United Kingdom That's useful for sending parcels and users can just put in their number and their postcode and the rest of the address fills in. But it's also quite useful for specifying a region, for searching for shops near me or parking or whatever. Access to that database of postcodes and locations is particularly valuable, and the post office sells it. The more valuable your data is and the more your business model says you should extract value from it, the more likely you are to want to have it closed and charged and paid for. You may also restrict how it's used. If someone gets hold of the postcode database, they certainly can't publish it or try to resell it. That reuse restriction will come in the form of license terms. Your next reading is going to be about different licensing and the effects of that.

At various stages in your database application, you will need to be thinking about the structures of your data.
Play video starting at ::22 and follow transcript0:22
Why and where are we concerned about structures? It may be in the programming environment, that's probably where you've done most about different types of structure. The things that you meet there are arrays, and objects, and floats, and other numbers, things that are used for telling the programming environment what to expect of the storage, and the treatment of those data types. In this module, we're going to speak at a much higher level of types of data structure. When we're modeling data, we'll talk about certain relationships, and ways in which the data we're trying to represent can be represented, how it should be brought together. It's also important to think about these structures for being able to serialize the data, that is to write it out either to a file or to a network connection in a way that can be read elsewhere. Because serial formats, a text file or some such way of passing it on require certain things. Although you can structure things in any way you want, ultimately, they tend to make certain structures easier than others. Exchange protocols, very similar to serialization. If you want to be able to exchange data with another application over a connection of some socket, then again, you'll have to think about the structures that match best for that. They have a lot in common with the serializations.

The other thing is worth bearing in mind is when you're communicating data with humans. That we, as users, have ways of structuring information that we are more comfortable with or less comfortable with. We'll also find that certain complexities, if there's complexity in a certain part of the data, we may want to structure the way we present it differently to make that easier to understand, and make that clearer. What shapes are we going to be talking about to satisfy those everything but the first layer? The first level, I'm assuming you've covered in algorithms and data structures elsewhere, and to a large extent when you're making a database that is handled by the lower level operation of the software. The other items in the list are more on the data modeling, and then we'll spend a bit of time on those in this module. Shapes we're going to be talking about are similar to the ones you find in the programming data structures, but this is slightly different. In the coming videos we are going to be looking at tables, which I'm sure you've encountered. This's one of the most general purpose, and intuitive ways of structuring data. We're going to look at trees, which give a bit of hierarchy to the structure. We're going to look at graphs, which are excellent ways of representing heavily interconnected data. We're going to look at the things are a bit harder to get simple information out of media such as sound, and video, for instance, or extended text content. We're going to also briefly mention objects, and documents, also where you don't look into them too much. The database itself does not understand that much of how they work. Over the coming videos, we're just going to review these very quickly, and then over the topics in coming weeks, we'll see how they are used in practice, and how the selection of data shape would inform the database paradigm that you wide up using.

When looking at data structures. And the first one we're going to look at is the table. We've you've probably encountered tables in previous database modules. You have encountered them in textbooks and all over the place, because they are by far the most common way of presenting data.
Play video starting at ::33 and follow transcript0:33
Now, as with a lot of computer terminology, we're going to be using the word table to mean something a bit different to the more generic idea of something with columns and lines, columns and rows. So I'm going to define a table for my own purposes. And sort of show what it is that I would expect a table to be and then we'll nail it down much more precisely in the coming topics. I think everyone generally agrees that the table has cells, that there are a number of rows, a number of columns. So the rose being the horizontal lines of cells and the columns being the vertical lines of cells. And I'm going to talk about a much more or a slightly more specific sort of table, Which is to say one in which every row represents a thing.
Play video starting at :1:36 and follow transcript1:36
So if we think about our database as giving information about things, so we have a list of things and we have some information about them. The row provides us with each row is a thing that we're going to give information to.
Play video starting at :1:53 and follow transcript1:53
And then in those terms, each column represents a type of information that we're going to give to each of those things.
Play video starting at :2:2 and follow transcript2:02

Let's take an example, because that's a bit abstract. This row is going to be about avocados.
Play video starting at :2:10 and follow transcript2:10
So by putting avocado at the beginning of the row in the first cell, I'm indicating for our purposes that everything that occurs to the right of it in this row is going to be about an avocado.
Play video starting at :2:26 and follow transcript2:26
I could say that if I put a heading on this column, water in grams, this is the amount of water contained by 100 g of whatever of something. So this column is generally generically about water content, just as the row was generically about avocados. And then we can have a cell that gives a data point a data value for the information that the column represents as it applies to a thing that the role represents. So in this case an avocado has 72 a half grams of water per 100 g. So we can add things to this data. So this cell here gives you the amount of water and butter. So each of those rows is a different thing, but they're still using the same column. So they're referring to the same data point. If we add another another category of information, we've added fat, then we can say butter has 82.2 g of fat per 100 g. So something this simple and clear is very obvious structure. So when I talk about tables at least for this topic, this is all I mean we'll get more specific when we when we get a bit more specialized and we start talking about relations. That seems pretty general purpose. You may be wondering what it is that I'm cutting, ruling out by talking about it this way. So let's have a look at what a table doesn't look like.
Play video starting at :4:18 and follow transcript4:18
Well this might be the sort of thing you might see in a spreadsheet. What this is is a load of information put into a tabular environment. But if you take a column, it doesn't necessarily mean a specific thing. I've reused the same column multiple times. A row doesn't necessarily mean a single category of data. I've put summary information underneath. I've used color and I've used formatting to help me work out what goes where, whereas for a table as I'm defining it, the thing that gives you the information is the row and the column and the data in that and nothing else. And I want to just give an example of a middle ground here. So this is a common way of getting open data in tables. This one comes from the un and what we can see looks very much like a table as I've defined it. Each row we have South Asia 2000 and five had a population of that and a value column that gives you 33.6 of the population were below the age of 15 in 2005. What makes this less tabling from this definition is that the value column doesn't really tell us anything. It might be a percentage, it might be a number of millions of people, it might be a ratio. And the only way we know what the value is, is by looking in the contents of the cell next door. So although this is kind of a table in the way they talked about it, It would look much more like one if I turned it into something like this. So because there's a limited number of years in there, I've given 2005 population its own column. And now I know that anything that goes in that column is millions of people In 2005. So whatever I put in the row is immediately interpret able if there were a lot of years, probably much tidier would be to have a separate years column back but still keep the categories separate. But both of these are much more like the sorts of table that will be meeting in databases in the in the coming topics.
Play video starting at :6:59 and follow transcript6:59
So one of the big strengths of a table as we'll see is that it communicates very directly. It's very easy for people to understand. It's very easy to structure in classical programming data structures and it's very easy to search, especially if it is structured such that each column behaves in a predictable way, and doesn't need too much context from neighboring columns. It

becomes very very quick to process and that's why it's the foundation of relational databases which will be spending the most time looking at. And however, it isn't necessarily the clearest way of structuring or communicating data. That branch is quite a lot that has more complicated hierarchies and has a greater level of richness. So in the next video we're going to be looking at trees that handle that in a slightly different way

A tree in computer science is the data structure that gains its logic in its language by metaphor from a real tree.
Play video starting at ::27 and follow transcript0:27
This tree has a root note. Everything comes from down here from one point, at each branching point, at each branching point we get multiple branches coming out. And if it weren't winter their belief nodes at the very end, any given node can trace back to one parent branch point and then another parent branch point and all the way down to the root.
Play video starting at ::58 and follow transcript0:58
And if I choose any two nodes, I can follow them back to a branch point that they have in common.
Play video starting at :1:9 and follow transcript1:09
So the structures that we used to encode a tree structure in computing whichever version of a tree may be, get their rules quite strongly from the metaphor of the botanical tree. There is another source though for the metaphor.
Play video starting at :1:31 and follow transcript1:31
So the concept and the structure of a tree in data science is largely derived from the biological tree that the metaphor refers to. But for our iconography and for our language we tend to go to the family tree, and not so much this sort of very literal family tree impression, but the slightly more abstracted one. So here's a very early perhaps less clear example. Of the genealogy here is the Dukes of Normandy, where you can already see that the and the key change in iconography, the flipping of the root node now being at the top has already happened. So the parents are all the top and the children go towards the bottom. We've taken our tree and we've turned it upside down and the roots are pointing up now and the leaves down.
Play video starting at :2:37 and follow transcript2:37
And we can see that bit more clearly in a modern version of family tree. This is of the Wedgwood's and that Darwin's and the Vaughn William's family, very, very large and influential family in England. As you can see lots and lots of family members, a certain amount of cousins marrying cousins. And you can see a lot of the imagery that we use in computer trees. So we can we have the the top nodes at the top, so the root node that everything derives from is at the top. Common parent nodes are towards the top and then as we get lower and lower, we get towards leaf nodes. No further nodes coming.
Play video starting at :3:25 and follow transcript3:25
And I'm trying to avoid, you may hear me struggling. I'm trying to avoid using specific vocabulary because it gives away the family tree origins. We talk about parent nodes, we talk about child nodes, a parent node of a parent node as an ancestor. So we'll be talking about the ancestors and the descendants of a particular node meaning going upwards or downwards in this sort of family tree schemer. So the vocabulary we use, the images we use all come from a family tree

and we've got to be careful and the reason that I'm distinguishing between the family tree aspects and the biological tree aspects is that in a normal family tree, if we look, for example, at Charles Darwin's parents here we have Mary Howard and Erasmus Darwin, there are two parent nodes in a family tree for most child notes, remember that for a most computer science implementations of trees, only one parent node per node. And this is why I distinguish between these two different sources for the metaphor of tree.
Play video starting at :4:47 and follow transcript4:47
How does this work in practice? Well, let's take an example of a work of fiction or a world of fiction in this case, the Harry Potter series by the author, JK Rowling. And so at the top of the tree, I've put Harry Potter series, this is the object that we want to represent. And then within that there's some knowledge about it that we want to put into the tree, we say the author is JK Rowling. and so we can have some branches and nodes that lead down to her.
Play video starting at :5:18 and follow transcript5:18
This series has parts and within the parts there will be Harry Potter and the Philosopher's Stone and the other books and stage works and so on. And the Philosopher's Stone has at least two versions. There's a book as a child node and there's a film as a child node and you can do the same for the games and so on. And the book We could give all sorts of information, we could have its title, its publisher. In this case, I've given the date is 1997 and this information comes as child nodes. You can already see, this is quite heterogeneous that the types of data that we are representing here, it can be very different. They don't have to be the same on any given level. So I've also given another child for Harry Potter and the Philosopher's Stone for the film and that has a child node for the date and the cast and the date is 2001 here, the cast, Daniel, Radcliffe, Emma, Watson and so forth. So a lot of very complex and rich information is here. There's quite a deep tree. There's not much repetition, there's not much of the sorts of things that you can often see in trees, but it should give you some idea of the sorts of structures if I wanted. I could put in book, we could have chapters, we could have pages, we can have all sorts of things, translations at this stage. It's quite a potent, powerful way of relating things in a contained fashion.
Play video starting at :6:48 and follow transcript6:48
So hopefully you've got some idea of the way that trees work now and the benefits for deep structures and for very heterogeneous data, where you don't really know how many of what things are necessarily going to happen at any given level on the tree and in what order.

Now, we've looked at tables and we've looked trees. These are the most common, most ubiquitous ways of representing data and communicating it. If we look at tables, if you've taken a databases module in the past or use databases, you have seen how they are powerful and can pretty much encode anything. You'll see tables in practice, in textbooks on any subject. In newspapers. They are the most intuitive way of communicating information. The tree is also not just very powerful, but very commonly used. If you've done Web development course, then you'll have seen it for HTML. That means that every website you visit would be represented at some level as a tree. Why would we need anything other than those two ways of thinking about representing our data? Well, let's come back to the trees to start with. Let's specifically come back to this tree that I gave you in the last video. Remember, this was the creative world of JK Rowling and the world in which Harry Potter, the character lives. We have parts of that come

from the various volumes Harry Potter and Philosopher's Stone, that becomes a book, becomes a film, that becomes a computer game. We would expect there to be other parts for the other books and the other films. Now, let's zoom in on a detail here. We have a film and it has a cost. The key requirement, one of the big, most important restrictions and definitions of trees that we've established is that any node can have as many child nodes as it needs, but we'll have exactly one parent except for the top node, which will have zero.

Play video starting at :2:32 and follow transcript2:32

The Daniel Radcliffe and Emma Watson nodes here, which are leaf nodes, can only have exactly one parent, which is this cast node, which is in the film node, which is in the Harry Potter and the Philosopher's Stone node. What happens when we add another film? Because what we'd want is for them also to be in the cost of this other film. They can't have multiple parents. There are ways round this within a tree structure or tree paradigm, but they are to some extent attempts to breakout of up your tree structure. Because what we have here is the beginnings of a graph structure. The graph is what you get when you remove the requirement for every node to have exactly one parent. What that does is it gives you either multiple hierarchies. This is an example of a multiple hierarchy where there's always an upwards direction, but there are multiple parents, or it can give you something with no hierarchy at all. For example, if we were interested in, in Emma Watson, more generally, we might build a whole structure around her whilst there's another structure around Harry Potter itself and another structure around Daniel Radcliffe and so on. We can center the graph at various points, but there is no trunk. There is no single route that everything comes from. This less hierarchical, decentralized structure, the graph, is one that's also incredibly common. It also happens in the Web. To some extent, it is the Web. If the Web it is defined by the links between Webpages, then that non-hierarchical network that gave rise to the name Web is in fact a graph.

Play video starting at :6:3 and follow transcript6:03

Beethoven's Sonata quasi una fantasia comes to us in a variety of different ways. It comes to us in the form of a notation, a special set of symbols laid out on the page that tell the pianist, in this case, what note to play and how to play the piece. You can think of them as an instruction manual or a cookery recipe. Then the other way we tend to experience this music is as a performance when someone realizes the notation at an instrument. In both cases, what we want is a way of retrieving music based on some aspects of its content. That requires us to represent something much more complicated and something much richer than a series of data fields and something that's very hard to summarize in a database structure. The same is true of a video, of a film, which might have a screenplay that gets realized by actors. In many of these cases we'll have an image, we may have a symbolic form, as in a transcription of the notation, we may have a sound wave or some other raw chunk of data that's very difficult to interpret. In the case of a piano piece, we may actually have midi or some other way of recording when keys are being pressed and how hard they're being pressed. But the types of retrieval we use for this and the way we interrogate that data is likely to be quite different to the way we handle other databases. It's also the case that the data sizes for any given field, for any given row in the database is likely to get quite large, quite quickly.

Play video starting at :8:26 and follow transcript8:26

Often when we are retrieving from media and other complex data, what we really work with is extracted features, information that is a partial view of some of the content of the media. What

are the data structures we've visited so far? We've looked at tables which we can say are the general purpose place you will go to first when you know you've got some stack of information to store. They're well understood, they're well-supported in software. Trees give you a degree more freedom especially if your data is naturally hierarchical, and we'll take some examples of that in future topics. You also encounter graphs, which also are quite good at dealing with this more heterogeneous data sources, which don't have hierarchical components.
Play video starting at :9:41 and follow transcript9:41
Then we've met this idea of rich data that you may not have very good ways into. They're essential just blobs of data. In those cases you will usually be extracting some features, deriving some information from that, that reduces the complexity to the point that we can structure it in a database. One thing I haven't discussed in detail yet is the situation where you have a list of things, but you don't really want anything clever from the database. If I have a list of quite heterogeneous data, I want it stored and I'll want to retrieve it based on whether a test I run at run time works. If it passes the test then I want it back, if it fails the test, I don't want it back. Don't try to index it, don't try to do anything clever. This may sound like quite a low intelligence, unsophisticated tool, but what it does give you is a much easier way of distributing that database more widely. Because if there aren't interconnections between your data and if you don't need to build sophisticated, complex rich structures into the database, it's much easier to break it into lots of pieces, run it on different systems, and get truly parallel, quick distributed operations. That's the menu of structures we're going to be looking at over the rest of this module. At this stage, we might be able to recognize them when they come in, but we're going to have to do a certain amount more work to be able to be in a position to decide how we want them for our own storage and for our own processing.

In this topic, and in the coming weeks, we're going to be looking in more detail at Relational Databases. The first question that makes a lot of sense to ask at this point is, what is a relational database? The simplest way to define a relational database that I could come up with was a relational database implements the relational model. There are several problems with that. One is that it's not necessarily clear what that means, and another is that it's not necessarily true. But let's start from it, because it provides such a quite a good basis to work from. This is quite densely packed, so let's go at it in small pieces. Model. What do we mean by model? Now it's a word that we're going to use a lot in this course and that I've already used a fair amount. What I mean by model is a way of abstracting complex real-world, usually data, into structures we can then reason about. We take messy things, and we find categories and commonalities and structures that repeat that we can then talk about and manipulate and represent. Now, modeling happens at multiple levels within a database application. In this discussion, we're talking about models for how data itself works, that is very high level, a very abstracted definition, and it'll be very generalized. Effectively, what we are going to be saying is a relational database is one that uses tables, and that's a very high level approach. We'll also be using model in terms of our data itself. We'll be using model for a very much more detailed, more complex, more rich set of structures. Models operate at all levels in Software Engineering and in data knowledge management and so on. In this case, as I say, what we mean by model is the way that we represent data itself in the abstract. We've got a model that's being

implemented by these database systems. What is that model? It's the relational model. The next question is, What does it mean to be relational? It all comes back from this word relation. What's a relation? A relation is approximately a table, especially in the definition that I used last topic when I gave you quite a clear idea of something that had rows, each row was a thing that we were modeling, and each column was an attribute of that thing. The list of things that were in a catalog or whatever would go as rows, and the attributes, the measurements, the observations about them will be represented as columns. In that sense, a relation is essentially a table or to be a bit more precise about it, a relation is the combination of the definition of that table, what the columns mean, and all the values that are stored in it.
Play video starting at :3:56 and follow transcript3:56
That is the core of the relational model. The relational models having defined that, make some rules for how we're allowed to manipulate those relations.
Play video starting at :4:12 and follow transcript4:12
I'm summarizing a set of rules created by code who founded the field. The first rule is basically that everything is a relation. That can be broken down somewhat. All operations that you can do use the relational model, all operations ask for tables, operate over tables. The data is represented and accessed at least from the point of view of any user as tables. It may have all sorts of internal manipulations that do other things, but from the point of view of any functionality that is exposed, these are tables, these are relations.
Play video starting at :4:58 and follow transcript4:58
Fundamentally it's not just the information that you can change using tables, but it's also information about the information. You can restructure the database, you can create new databases, you can create tables, you can create users, all entirely by manipulating relations. It's not enough for a database that have some relational functionality. The entire database must be accessible through tables.
Play video starting at :5:38 and follow transcript5:38
It goes further than that. To some extent this must be a robust abstracted system, and what that means is that if the hardware changes, it should make no difference to any of the functionality. If the operating system changes, again, the user of the database should not recognize the difference. Disks are replaced. It doesn't matter what the formatting of the disk is from the point of view of the application, for the point of view of the database management system. It should be identical. In other words, it's a model. For relational database to implement the model, it must behave consistently no matter what they physical and software based constraints are. Similarly, if the data is distributed across a network, it should look exactly as it would if it were concentrated in one place. This is a summary of a set of rules for how a relational database work, and it's worth saying that there is a certain amount of debate about how this should be interpreted and how they should be used in practice. But nonetheless, the founder of the relational model also read the rules.
Play video starting at :7:8 and follow transcript7:08
Now we're going to be meeting effectively three different models over the course of this topic alone. The relational model, which is an abstract model and about which a lot of reasoning has been done to prove its behavior and to prove its reliability. There is also a second thing, the entity relationship model. These are not the same, and the word relation in both means something different, as we'll see shortly in the next video. Now, I'm not introducing these two

concepts at the same time to confuse you, that's not my intention. Entity relationship models are used very commonly for modeling all things. Here we go. An ER model helps model concepts. We use them to model data structures in other contexts as well, but they're very, very commonly used for modeling relational databases. I will be introducing the ER model and we'll be showing how to draw an ER diagram. But you have to be very careful and keep in your mind that the relation in the relational model and the relationship and the entity relationship diagram are different things.
Play video starting at :8:32 and follow transcript8:32
Another thing that I, again, don't want to confuse you about, but you have to bear in mind as we go through this course is that the relational model has almost never been purely implemented. What we'll be talking about overwhelmingly over the coming topics is SQL, sequal. Sequal is a partial implementation of the relational model, it's not very strict, and it has a few departures from it. But it's close enough to the relational model but much of the theory underlying the model can be applied to SQL, to sequal and therefore those packages that implement some or all of sequal, MySQL, PostgreSQL, SQLite, and so on.
Play video starting at :9:30 and follow transcript9:30
Those packages are close enough to SQL that they can be taken to have the properties of SQL, and SQL can be taken to having many of the properties of the abstract relational model. Even though at each stage there's a few impurities creeping in. You'll notice I say sequal and SQL interchangeably. It's very careless with me. I tried to be consistent, and I apologize that I won't be. Over this topic, we are going to be meeting those three models, the relational model, the entity relationship model for drawing and planning structures, and sequal itself for actually enacting those structures and for implementing a database.

We've designed our database, we've sketched it out, and we need to build it. The next thing we have to do is speak to the database management software to make the database, to make the tables, put the data in, and then query it. The language we're going to use to do that is SQL or SQL. You'll notice me quite often saying SQL. I think it's probably slightly more common that people say, "SQL." That's just a matter of habit from when I first read it, rather than one being better than the other. But lots of different databases implement SQL to some degree or other. Even access gives you the ability to use it to a certain extent and for certain operations. But a lot of databases are fully controllable using the SQL standard or something very close to it. We will be using MySQL in labs on MySQL. But lots of other packages exist, many of them Free. Postgresql, for example, and SQL Light, which tends to get packaged in other programming languages. All of them broadly will have the same commands with the same syntax and there'll be small differences. Only that you'll have to sort of look up in the, in the documentation for that particular flavor, a particular implementation of SQL. We're going to be speaking to databases. We're going to be using SQL. The theory around relational databases and around SQL distinguishes between talking about structure. So creating the databases, the definitions of tables, and manipulating the way the database works. In that context you have commands like Create, Drop, truncate, alter, and various others. Then talking with or about data within those structures. Those are commands like select, insert, update, and delete. Now that's the only time I'm going to distinguish, I think in this entire module, between what's called the Data Manipulation Language and the Data Definition Language. It's something that's a part of the

theory that shouldn't really trouble you too much in practice, the two sides of the language operate largely in the same way. But let's look at some examples. I'm going to start with select, which is the command you use to retrieve information from a table of tables.
Play video starting at :3:8 and follow transcript3:08
Although the interpreter for commands is case insensitive in almost all cases. I will exclusively use capitals for keywords and most other people do, largely because it helps to see the structure of a command. In this example, select planet name from planets, select and from special SQL keywords. I've put in block capitals. What this is doing, the Select word at the beginning introduces a query. It's all it does. After the select, you have what's called the column specification, which is where you name the fields you want to see in the results table. Because everything that comes back will be on a table.
Play video starting at :4:6 and follow transcript4:06
The name of the columns in that table will be derived from this list. But also it tells the system with where to look, what the name of the column in the source table is. Planet name is the name of a field in one of my tables. It will be the name of the field in the table that is returned to me. Then after we've set the column specification that we have from, which tells us where to get these fields from, where to get the columns from. In this case, just one table.
Play video starting at :4:44 and follow transcript4:44
The overall thing is get planet name from planets for all the entries in the planet's table, and return it in a table which just has planet name for every case. That's relatively simple "Select" query.
Play video starting at :5:7 and follow transcript5:07
Let's give it a constraint. Now, we're going to say, "Only return me the planets from the planet table who have a day length longer than 200 hours." But I don't care about their day length. If their day length is longer than 200 hours, give me the result, but all I want to see is planet name. If I wanted to see planet name and day length, then instead of select planet name from planets where day length is greater than 200, I'd go select planet name comma day length, and then the rest will be the same. I have to explicitly name all the fields I want to see separately from what I want to build my constraints, my restrictions on. That's a quick introduction to the querying, to the select command. Don't worry, we'll do some examples before the first lab. Then I just want to introduce a command from the Data Definition Language, "Create". "Create" is a slightly more complicated command. You can use it to create different things. After you've said the "Create" keyword, you then need another key word that tells it what the new structure is going to be. In this case, we're going to create a table, so "Create Table". That combination tells that it's going to be a table and you immediately follow it with the name of the table. Create Table planets, and tells it to create a new table called planets. Then the rest of the command goes in brackets; open brackets and then the names of the fields, the columns that we want to go into this table. Planet name, day length, year length, and you separate them with commas. But you do have to tell it what data is going to go into those fields. After the name of each column, we're putting in the type of thing. Char or char is character data set text, and then in brackets, how long that texture should be. In this case, I've said no planet name is going to be longer than eight characters, so char open brackets, eight, close brackets tells it how much space to allocate and that it's going to be text. Planet name, char, open brackets, eight, close brackets, comma says that the first field will be planet name, and it'll be text. The other two fields, day

length and year length are integers. I'm only specifying them to the nearest whole number. There's a whole suite of different number formats that number data types in SQL and in the separate database management systems. You'll have to look those up to work out the best one. It'll only matter probably if you are going to have a big table because the amount of space reserved for an int versus a small int, is relatively irrelevant for most users.
Play video starting at :8:39 and follow transcript8:39
We've named the fields that are going to go into the table, and strictly, we can end our "Create Table" statement there. If we do that, we will definitely have a table, we will definitely have a relation, because there's a constraint on relations in relational model of databases that every row has to be unique. We can't have a duplicate row whereas a table can have. The way to enforce that a table doesn't have duplicate rows is to specify a field or combination of fields that will be unique. We do that using primary key. Primary key says, in this case, that planet name is going to be a field that will only happen once, so the name Mars will only happen once in this table, the name Earth will only happen once in this table. We specify there the combination of columns that will ensure that a given row will be unique, and that can be tested for by the system and we can ensure that what we have is a relation as well as being a table.

We've talked about using relational databases, and in the lab that's coming up we're going to try actually playing around with one. In the lab environment, there's a MySQL instance, already installed and running. We're just going to try creating some things and exploring a bit. I thought I'd give a quick run through, so you can see it in practice, before you go and do it for yourself. I'll make this a little larger so that it's nice and clear.
Play video starting at ::49 and follow transcript0:49
In this lab we'll be using the terminal to interact with the database. Just make that a bit bigger. That's as big as it wants to get right now. No, and it's gone.
Play video starting at :1:8 and follow transcript1:08
The MySQL process can be started with a simple command. There's a number of things that you would usually put after MySQL here. You'd usually put your username and say you want to login with a password, and you might also say which database you want to use. You can tell that from the command line, but I'm going to go straight in with just this command. The labs have been set up so that you can log in without a group password. I'm here and the first thing I can do to check the state of the database, is to, say share databases. A database management system can have multiple separate databases in it. If I have a look here, what I get is information schema, MySQL, performance schema and sys. These are the tables and the databases that the management system uses to manage, the actual databases with. Set for the relational database is one that behaves entirely like it consists solely of tables. All the management of users and of data, has to happen via tables, or at least has to present that way to us. There's a user's table in here which will say all the users and their permissions, and so on. Everything is tracked through databases and tables. But none of these is a table that I want to interact with. I want to create my own database and I'll do that using the create command, create database. In your case, you're going to be doing the planets and moons example. In my case, I'm going to do a degree program, or a [inaudible] department, let's say. Create database degree. Now, I can go straight from that into typing commands, but if I wanted to select things or interact with tables in that database, I'd always have to say which database, I was talking about. I could do select staff

from degree.table name, whatever the table is called. To avoid having to specify the database each time, I can say use degree. This now puts me inside that database and all the interactions I have with tables, will be with tables of that database. Let's create a table.
Play video starting at :3:51 and follow transcript3:51
Create table I almost typed a table [inaudible] and I'm going to call it lecturers.
Play video starting at :4:6 and follow transcript4:06
I'm not egocentric, but the first data structure I'm going to create is going to be me.
Play video starting at :4:14 and follow transcript4:14
It's going to have- how shall I do this? Let's have a name-
Play video starting at :4:25 and follow transcript4:25
there are various different text types in SQL. I'm using Varchar here, which is variable length, and I'm specifying that it can't be more than 255 characters. Hopefully that safe, and date of birth and that's going to be a date.
Play video starting at :4:51 and follow transcript4:51
Date joined maybe, which is also a date. Now on the assumption that name isn't necessarily unique, and because I haven't created a lecturer ID which is what you saw me pausing and thinking about before I acted. I have to have a combination of fields from my primary key. My primary key is going to be name and date of birth. Now, there's no absolute guarantee that that is unique either, but the probability of a clash is much lower. For the sake of a demo, I'm going to do it like that. Now I have a table, if I go show tables, remember we had show databases, we can see the table. I can see a bit more detail if I ask it to describe lecturers.
Play video starting at :5:54 and follow transcript5:54
When I do that, I can see a bit of detail about those fields, and I can insert a value. Insert into lecturers and values, David Lewis. What date of birth shall I give myself? I'd quite like to retire soon, so maybe if I put my date of birth as 1955, this is not my real one, in case anyone's wondering. When did I join this institution? Let's say I did it so early, it was in 1985, on the 5th of March. Unusual time to join a university but nonetheless. I've inserted that data into the database. Now if I select the name from lecturers, and get the case right, there I am. I can see all the fields, if I replace the specification of a column name with the star, and now I get all of it.
Play video starting at :7:19 and follow transcript7:19
That's just a very very quick introduction, you've got instructions in your lab sheets. Go and have fun with MySQL, and I'll see you afterwards.

towards the end of your lab, you were asked to make a query that used to tables rather than just one.
Play video starting at ::21 and follow transcript0:21
And the mechanism for doing that for linking to tables is called a join. And in this video we're going to be talking about joints.
Play video starting at ::34 and follow transcript0:34
So let's take the example that we had in the lab.
Play video starting at ::39 and follow transcript0:39
Planet has a name, the moon has a name, and any given moon has a planet.
Play video starting at ::47 and follow transcript0:47

And we can show that as a relationship using the entity relationship diagram.
Play video starting at ::55 and follow transcript0:55
Actually, more realistically what we had, in fact was a load of extra attributes as well as just name, so that we could query on them as well. But it's really this has planted the relationship that we're going to be talking about in this video.
Play video starting at :1:14 and follow transcript1:14
The way the first sort of join you can do is to connect the tables simply by listing them in the from clause of your select statements. They select Lead name, Rhythm named, Bass name, Drums name, From Lead, Rhythm, Bass, Drums. So say I have a table each for musician friends of mine, all the people I know who can play Lead Guitar. All the people I know who play Rhythm guitar. All the people I know who played Bass guitar and all the people I know who can play Drums. And I want to see what possible band make-ups I can have from that set. And then doing this. Query will automatically do that for me, will combine all the tables and give me all the results and this is good a Cross join.
Play video starting at :2:6 and follow transcript2:06
And the important characteristic of a Cross joint is it will combine everything, so every Lead guitarist will be combined with every Rhythm guitarist and so on. So I'll be able to see all the possible band make-ups, that I could I could have assuming I want to quartet. So let's imagine I've got seven Lead Guitarists, 12 Rhythm Guitarist because that's easier. 8 Bassists and 15 Drummers.
Play video starting at :2:38 and follow transcript2:38
If we use a Cross join, I'll get the multiple of each of those seven times 12 times eight times 15 rows in my results table. So for just a small number of musician friends, I'll have a results table with 10008 and 80 entries.
Play video starting at :3:1 and follow transcript3:01
So a Cross joint is something to approach with a certain amount of caution. It's not used very often and if it is used, it has to be used with a lot of constraints in the where clause to really reduce the size of each of the component tables because Cross join gets very big very, very quickly. And any operations that are done on the results table, such as sorting or filtering will take a long time because they may be done in Memory or two Disc, which on a big table will be a problem.
Play video starting at :3:41 and follow transcript3:41
So that's the simplest but possibly the least useful of the joints. Let's try another join. And this is the one that we had in our example. So I've changed the field naming convention slightly to give you another example of how to do this. In the lab example we had planet name, a Moon name as as fields so that no matter what table you were in, they were always distinct in this case I've said planet dot name. So the dot specification tells your table. So planet dot name. In my select here means give me the name field from the table. Planet. So select planet dot name comma moon dot name comma has liquid water. So this is a bit closer to the to the entity relation relationship diagram that I showed you and it's not necessarily a better way of doing it. Just a different way of doing it. And again in the front claws. All I've done is list the tables, but I've given an equals constraint in the where clause where planet dot name equals moon dot as planet. So there's a field in the moon table that is identical with a field in the planet table and that constrains the table joined very very tightly.

Play video starting at :5:12 and follow transcript5:12
And this is called an Inner join. Now, this way of specifying was is fine. It works because your database management software will recognize that it's an Inner join. It will not do the Cross joint first and then apply a filter.
Play video starting at :5:36 and follow transcript5:36
But another way to specify that's clearer, that we're using a different sort of joint is actually to say in the from list of tables from planet, join in a join moon.
Play video starting at :5:50 and follow transcript5:50
So from planet, inner join moon on planet name, it was moon dot has planets. So what we specify is the name of the first table, the type of join, the name of the second table and then on we say exactly what the criterion is from making that combination.
Play video starting at :6:11 and follow transcript6:11
And if we do that, the numbers get a lot smaller. So assuming we have 8 Planets And 173 Moons.
Play video starting at :6:22 and follow transcript6:22
If we had a Cross join, we'd have 1,384 results in the results page. They'd also be meaningless because every Planet, so I'd have Earth having phobos and Saturn having our Moon and so on, so it wouldn't be meaningful. But it would also be a big table. He's in the Inner join only the Moons that are actually associated with the planet would be shown with that planet. And the result of that is 173 results, which is substantially fewer. And as I say, in most cases much more helpful. And the optimizer built into your database software will be able to analyze the quickest way to carry out that joint.
Play video starting at :7:9 and follow transcript7:09
And the other type of joint I'd like to talk about briefly is what's called an Outer join.
Play video starting at :7:16 and follow transcript7:16
So in this case, all I've changed is instead of saying in a join in the from clause, I've gone from Planet left, Join Moon.
Play video starting at :7:28 and follow transcript7:28
And what that is saying is, I want all the rows from the left, whether or not there's a match and then only the rose from the right where there is a match. Whereas previously the Inner join takes only the rose from either side if they have a match. So only include the Earth if it has a Moon in the other table and only included for that row. So Mercury will not appear in the results table because it doesn't have a Moon. In this case, we're saying I want all the Planets no matter what. And when I do that, remember, we had 8 Planet 173 Moons. The Cross joint produced 1000. The Inner join Produced 173 records. All this does is it adds a few records. And what those records are? Are these Null Records, Null entries for the two Planets that don't have Moons. So it preserves everything from the left hand side. And then gives them blank values where there's no no match. And that can be useful, especially for finding those examples where there is no match.
Play video starting at :8:47 and follow transcript8:47
So to recap, we've had ways of joining tables that affect the number of rows and the sort of contents you get in the results. We've met the Cross join that combines every row from each table with every other row, producing a very, very large result set. We've met the Inner join, which only includes rose from each table that match each other in a particular constraining way,

and then the Outer joins, which might be left or right. That always include all the rows from one or other of the tables, and then the matching ones from the other table, and those are the three sorts of join that will be meeting most. And we'll explore how to use them in future sessions.

Joins are an important part of how rich relational databases work, and also part of how entity relationship models often work. We've explored some aspects of how joins work and the types of joins. In this video, we're going to explore a few more. Particularly, we're going to look at what's called cardinality. Cardinality is concerned with how many rows in each of the tables that participate in a join match with how many rows in the other table. When we were looking at cross joins and at inner joins, we were talking about how the constraints affect the tables that are being connected together. Cardinality is often expressed in terms of a ratio. One to one, one to N and so on. One-to-many, you might say. I'm going to talk about three types broadly. You can have one to N relationships where one row in one of the tables is related to some number of rows in the other table. It might be zero, it might be one, it might be many, but we can't prescribe, we can't define it in advance how many there will be. The example that we've seen already of this is the planets to moon join. Where some planets like mercury, don't have any moons. The earth has one moon and other planets like Jupiter have many. We know that a moon can only orbit around one planet. But, we don't know how many moons there will be around any given planet. That means that for any one planet, they'll be N moons. N may be zero, it may be one, it may be many. The second cardinality that we're going to be talking about is a one-to-one. This is actually remarkably rare as a proper cardinality of join. In the example I gave of a pop quartet, there was one each of the lead guitar, the rhythm guitar, the bass guitar, and the drummer. But at the same time, in many cases where you do something like that, the makeup of a band might vary. For example, you might have two bassists or two rhythm guitarists. It's only in quite often, quite artificial examples that we have these one-to-one relationships happening. A slightly more realistic example might be a student to their final undergraduate project. There's usually any given student has only one project. Then that might be an example of a one-to-one relationship. I'm sure you can probably think of examples where even that example breaks down. These are rare, but they do happen. The other cardinality that we really need to think about is many-to-many. We use different letters for this, in the ratio we say M to N. Just to illustrate, that the numbers aren't the same number. There's some number of rows from one table that get related to some number of rows from the other table. We've used an example previously of student having a tutor. In most courses, in most programs, students will have multiple tutors at different times for different roles. In that case, a tutor can have multiple students, and a student can have multiple tutors. That's an example of a many-to-many mapping. Why should you care? What is it about cardinality that makes it worth considering in these contexts? Well, most importantly from our point of view, it's the implementation that changes and the implementation becomes quite different. Let's take them 1:1 again, 1:n or n:1 because depending on the order in which you have the tables, that can be either.
Play video starting at :4:47 and follow transcript4:47
We're going to have a strategy of simply using a foreign key. Putting the primary key of one of the tables as a field in the other table. If I were to draw this in the entity relationship diagram, just to remind you, Moon has planet. We have a relationship here where there are many moons and we can draw this relationship. Here you can see on the diagram, I've put m for many on the

side of the Moon and one on the side of the planet to indicate Moon always has one planet but the planet can have multiple moons. This is how you draw cardinality and the entity relationship model or one of the ways of drawing cardinality. Another thing that you can indicate using the entity relationship diagram is that the Moon will always have a planet. There is no possibility of a Moon not having a planet. It wouldn't be a Moon in that case and you can indicate that by using a double line to show that its membership is compulsory. Here we have a one to n relationship in our entity relationship diagram and we now want to turn that into a table join and we've already seen how you do that. You declare the table for moons, that's the one that has the n rather than the one in the ratio. One planet, many moons. It's the moon's table we have to change and the Moon table gets planet name in its definition so that we can relate it to the other table. We can also declare that relationship explicitly. You can add after the declaration of the primary key, a foreign key declaration and what that says is, the field called planet name in this table is actually a joining one. It's a reference to an equivalent field in another table, in this case planets, planet name. The whole sequence of the syntax series, foreign key, open brackets, the name of the field in this table, in this case planet name, close brackets, and then references, then the name of the other table planets and the name of the field that matches planet name. This tells the database management system that there is a join here and that it is potentially a one-to-many with this table being the one that has the many.

Play video starting at :7:38 and follow transcript7:38

That's how to do a one-to-many, that's probably the most common join you'll encounter, 1:1 as I say is relatively rare and quite often, it's a simplification of something that might be a 1:n. Sometimes you might just as well implement it as if it were 1:n. In the case of the student to project, it looks very much like the planet to Moon example. Project is by a student and one way to implement it would be just to put, to say actually the project is, if there's a 1:1 relationship between the project and the student, want to put them all in the same table because the student will have one project, it can have its own rows and we've solved that problem. We'll see in the next topic some reasons why you might not want to do that and that then means that you'll end up implementing it as if it were a 1:n relationship, but putting the primary key of one of the tables into the other table. In this case, the student and the project title are both in, so we have projects table, we have the student named in there as well. We can even make the student the primary key if we know that it's a one to one relationship with the students table.

Play video starting at :9:15 and follow transcript9:15

If it's the case that for example, a student who fails the project gets to retake and choose a new topic, then it's no longer a one to one, it's one to N relationship and we might want our primary key here to be based on the student and their title.

Play video starting at :9:35 and follow transcript9:35

This structure is easily modified to be a one to N. These are relatively simple cases. The one case you have to be aware of is the many to many relationship. The reason you have to be wary of it is it's very easy to draw in the entity relationship diagram. The entity relationship model is very comfortable with the many to many relationship. But that relationship is not modifiable directly in the relational model. When we turn it into a relational database, we have to cheat a little bit and we have to add a new entity into the system. Let's have a look at the example. A student has a tutor and students can have multiple tutors and tutors can have multiple students. This diagram here shows it in exactly the same way as all the other cardinalities we've looked at

by just putting an m or one in the place where the multiplicity is and you can immediately say multiple tutors have multiple students. But if you think about how we've implemented the one to n relationships in the relational model, you'll see what this doesn't work because if we add a field in say, student for the tutor, how many fields do we need to add for multiple tutors? It's fundamental to the relational model that these individual fields in the relation, individual columns are simple, they're atomic, they can't be a list.
Play video starting at :11:21 and follow transcript11:21
What do we do? We add a new entity. Now we have a special tutor role which might be done by module or might be simply free form. The point is that in any given tutor role, there is one student and one tutor. A tutor can have multiple tutor roles and a student can have multiple tutor roles as in, can be the student in multiple tutor roles but each one of those is a one to many relationship. We know how to model one to many relationships so this one works. Let's look at that in practice, what we do is have what's called a link table. Assuming we've got a student's table and a teacher table, we make a new table called tutor role where the student name or ID has a field and the tutor has a field and the primary key is based on the combination of the two. In this case, we can link from there outwards. We've turned a many to many into one to many, many to one pair of relationships and that solves the problem but the point here is that you will have to modify your entity relationship diagram before implementing the database to accommodate the restrictions of the relational model.

In this video, I'm going to be introducing the topic of database integrity. One way to look at this topic is simply as an answer to the question, what could possibly go wrong? If we can plan for all the things that could possibly go wrong, we can try to make sure that at least some of them won't. Although at least we have some structures in place that make it harder for particular patterns of error to happen. Let's get back to the planet's example, the planets and moons, and have a look at what data anomalies and data problems could happen. Well, we had this entry here that the moon Deimos is a moon of the planet Mars and has an area of 495 presumably square kilometers.
Play video starting at :1:12 and follow transcript1:12
What could go wrong? Let's say when I was typing this in manually, I put that it was a satellite of Mers rather than Mars. Now we've labeled a planet that doesn't exist, and depending on the query we do that means the moon may disappear from some queries. Also, when we ask questions like, how many moons does Mars have, we're going to get the wrong answer. What other problems might happen? Well, I might have forgotten to specify what planet Deimos circled around, and a reasonable default that are database may do is to use a value called NULL. Now, the relational model specifies a special logic called ternary logic. Instead of a Boolean of true and false, the SQL relational database specifies that we need a third value now for unspecified or unknown data. Here, if I forgotten to specify the planet name, we may wind up with again a planet name not only that is wrong, but will never align with anything. NULL is equal to nothing. If I try to do a join to count the moons, or to look at the relationship between moons and their planets, where Mers might conceivably appear NULL will never appear. This would also cause all problems in summary tables where I haven't provided the planet name. What else might go wrong? I might specify accidentally hit minus 495 for the area of the planet. In that case, it's not something that's obviously going to break the database because the area is

not connected directly to anything, but any calculations I make go horribly wrong. It's a problem that we can predict because we can actually know that a value of minus 495 is meaningless in this context. A negative area has no meaning in the geometry of planetary science. Knowing what values would be valid allows us to do something about it. Whereas if the area was put in as 95 by accident that's a typo, it's much harder for us to correct for that. If the value we've put in is meaningless, so we've had three things, no value has been put in, something that connects to another table has been entered incorrectly, and a value has been entered incorrectly in a way that renders that attribute meaningless. In this instance, we've got two values given. The same moon, the same planet name, and two different values given for area. We've still satisfied the rule that every row has to be different and include different values because the area is different here, but these are now contradictory. What is the area of Deimos, the surface area? We can't tell. We can put inconsistent values. Now, there are ways to protect against that, and the database that we created earlier will almost certainly reject one of these values. But say I have a typo as well, I've named my moon incorrectly, Dimos in one example, Deimos in another example, and given different area values. Now there's no way for the system to recognize that something has gone wrong. Here, I've misspelled the name of the moon and I've got a typo for the area. This is another of many possible errors that we can have based at least partly on human error. Let's have another example. Say I've got this mistake in my database. In my list of moons for Mars, I now have Deimos, and Dimos, and Phobos, thanks to a typo. All the data for Deimos and Dimos is the same still, because they are actually the same thing but I've just typed it in once and they're not noticed I've done it. When I typed it in the second time because of the typo, because I missed type the name, it didn't get picked up. Now, when we look at some of the data is going to be wrong. Now let's say that in the planet's table, I've also got an attribute for the number of moons. Mercury has zero moons, Venus has zero moons, Earth has one moon, and Mars has two. But now there's a problem, because if I were to count the number of moons associated with Mars, I'll come up with more than two. We've got two values in the database. One, a hard-coded moon count which says two, and the other, the result of counting how many moons we've got specified that gives us three. In this case, we've got incompatible values in different parts of the database. Let's take another source of problems. Say we were talking about Pluto. Now, Pluto has one moon, we've encoded that in moon count, and we've provided the moon in the moons table. That is Charon. Charon has just as Charon, planet named Pluto area five. But Pluto is no longer considered to be a planet so, alas, regrettably, we delete Pluto from the planets list.

Play video starting at :7:46 and follow transcript7:46

But now, we have another moon that is associated with a planet name that doesn't exist. Earlier on we had the situation of a moon associated with a planet that didn't exist because I mistyped Mers for Mars. In this case Charon had its planet name correctly specified, but I've deleted the entry associated with it from the planets table. It's an orphan record just as Deimos was in that first example. But it's an orphan record not as a result of human error, but as a result of an action performed within the database structure that's deleted the thing that it refers to.

Play video starting at :8:34 and follow transcript8:34

Now, that's a quick tour through quite a lot of different types of anomaly or error. But the questions we're going to be asking in the coming video and to some extent in the next topic are, to what extent can we find problems or patterns of problems that we can build in detection and

avoidance into the structure of our database, and which of these are human errors that are only policeable through checking, double-checking, and having extra review on a human level?

in the last video, we looked at some sources of error that can give inconsistencies within any database system. And some of those are avoidable only by carefully checking all the data that goes in, but others are, in some sense structural. And there are things that can be policed are enforced by the database itself.
Play video starting at ::37 and follow transcript0:37
So in the next topic, we're going to look at some theory and structures around avoiding those errors. But in this remains of this topic, we're going to be looking at just one very simple family of issues that can be dealt with. And those are integrity errors and constraints that we can apply that will avoid those through SQL commands. So this is what we call Integrity constraints. And these constraints can be applied to tables, relationships between tables and two columns. And what I'm going to do in this video is recap over some of those errors that we looked at in the last video. And ways that we've already met of reducing the risk of those errors or avoiding them completely. So let's take that first example where we had the moon Deimos and I'd mistyped the planet that it was associated with. So I associated with mass instead of Mars and it now vanishes from some of the summary tables.
Play video starting at :1:55 and follow transcript1:55
Now, we can say when we define the data stretches, when we're drawing the entity relationship diagram that it doesn't mean anything for a moon to be orbiting around either no planet at all or a planet that doesn't exist.
Play video starting at :2:14 and follow transcript2:14
A planet must exist in our planet's table for a moon to be meaningful to have a moon that orbits around it. And because that's a structural sort of generalization, because that reflects our model, we can embody it in the definition of the database system itself. So we can use the foreign key that we used before. And when we specify that and insert with the wrong value or an update that changes a valid value to an invalid value will fail and we'll get an error. So this code that we've seen before in our create table when we're creating the moon's table, if we say foreign key planet name references planets, open brackets, planet name, close brackets. As if we specify that this field in this table, the planet named one refers to the matching field in the planet's table. This will now be enforced by the database. So if we try to insert something that isn't in the planet's table into. So if we try to insert into the into the moon stable a reference tumors or to null or to anything like that, the insertion will fail and will get feedback immediately.
Play video starting at :3:45 and follow transcript3:45
Obviously to some extent, insertions into databases are usually made by other pieces of software. So if we built a complex web application that's doing the insertions, then we'll need somehow to propagate that error to the user. So they're aware of what's gone wrong. But at least the structure of the database is preserved by this integrity constraint.
Play video starting at :4:8 and follow transcript4:08
So keys are very, very useful as ways of preserving the integrity, the correctness of a database.
Play video starting at :4:18 and follow transcript4:18
We had an error where the area value was meaningless. So I'd accidentally typed a negative number into the surface area of the moon. If we know what is and isn't valid as a value, we can

put a special sort of constraint called a column constraint into our table definition when we define that column. So for example when we're specifying area before we said area isn't to say that it's an integer, but we can be more specific. Now we can say area into check open brackets, area is greater than zero close brackets. And now again any change or insertion that would create a value for area that fails that test that is not greater than zero will throw an error. And that can be quite sophisticated logical check it can check the value against other values in the insertion. And it can be made of multiple clauses. So if we knew that a moon was defined as having greater than a certain surface area, then we could insert that there. Or if we had different categories of moon, small moon, big moon, we could have it checking against those whether the area was within a particular numerical range. Several of the errors that we explored in the last video were inconsistency related. That is somewhere within a table or within the database. two values were stored that contradicted one another and that makes no logical sense.
Play video starting at :6:6 and follow transcript6:06
Now, the first example of those was when I put two entries in for Deimos the moon, and gave each a different surface area. So now we have two pieces of information to have equal status but which can contradict one another and we don't know which one is right. Again, this is a structural error. That is to say, when we start designing the data structures the model, we can say there will only be one entry for any given moon and each moon has a unique name. And we did that in our lab specification when we were building the table, we said that moon name was a primary key.
Play video starting at :6:57 and follow transcript6:57
And because of that, my second attempt to insert a row for the moon Deimos would have been rejected with an error. And that role of the primary key is as an integrity constrainer, to make sure that we don't get inconsistent values is really important and really useful. And this brings me to a general point, which is, you'll see quite a lot of examples of database design where the primary key is an automatically generated ID and that's often quite a useful thing and we'll see some examples of that in the next topic. But if your key is automatically generated, you lose the benefits of being able to make these sorts of integrity constraints or rather you have to do that explicitly as a separate stage. So, for example, if I know automatically numbered every new entry, then Deimos would appear happily twice in the table with a different ID. And it would simply conclude that they were different moons of Mars with the same name. So if we have knowledge of the uniqueness of any field or combination of fields, it's worth declaring them either as a primary key or some other sort of unique key. Because that constraint can be enforced by the database itself, rather than requiring us to do extra work on the data cleaning and data checking. Now, another example of inconsistency in the database that we looked at in the last video, was when the a number of moons was specified in the planet's table and could be deduced from the moon's table. So we accidentally had three moons listed in the moon's table, but we still said there were only two moons in the planet's table. Now there's another issue of how you make sure that only right information is in the database and the database only has correct information. But from the point of view of consistency, the way to avoid that error is not to store calculatable values. So having a field that was manually entered that said, the number of moons created the risk of inconsistency.
Play video starting at :9:30 and follow transcript9:30

Now in this case that was the value that was correct. So you could argue for using that as a way of checking the accuracy of the numbers in the database. But from the point of view of making sure that your databases itself consistent in what it's asserting, you should avoid having calculated values. And then again it becomes something that the database can ensure. And when you want to know how many moons there are, you simply tell the database to return the count of how many means there are. We saw another problem with the relationship between the moons and planets table, which was that Pluto has a moon caron. But Pluto has been removed from the list of planets. So we ran a delete command on the planet stable to remove Pluto. But now we had another orphan record. We had caron with a planet name of Pluto. If we inserted it knew it would have been rejected because of the foreign key constraint. But the change to remove Pluto was made after we've made that insertion. So there's no command to associated with, but still this is a structural constraint. This is something we can model and we can understand it makes no sense for a moon to have no planet. And so we can pre specify what should happen when a planet is deleted or changed.
Play video starting at :11:11 and follow transcript11:11
And the way we do that is using an extended form of the foreign key specification. When we make the foreign key command, we say foreign key planet name, references planets, planet name. And we add one extra clause on delete cascade. And what that means is that when the planet is deleted from the planet's table, it will delete all the entries in this foreign key table that match. The reason there's an extra command here is that the database system will know that there is an integrity issue but it won't know what to do with it and there are multiple options. So if instead of deleting the entry, you wanted to reject the deletion command. So when I try to delete Pluto it gives me an error and says no you can't delete it because there are dependent records. Then that's also an option.
Play video starting at :12:15 and follow transcript12:15
So again, we can embody the logic of our model in the structure of the database and then allow the database system to enforce it. And that takes a lot of responsibility office as software developers to build these sorts of checks in.
Play video starting at :12:36 and follow transcript12:36
The extra aspect of this that will be exploring in the next topic is this thing called a functional dependency. So we want to remove repeated information because we want to remove the risk or reduce the risk of inconsistent data across the database of anomalies that make the database less accurate or reliable. And there's a chunk of theory around that and that's what we're going to be looking at in the next topic.

Welcome back. So in this topic, we are going to be asking ourselves again and again the question, what can possibly go wrong? So we're going to be looking at the possible sources of error and of damage to our database and what we can do to mitigate against it. So what can go wrong? Well, we could get bad data coming in and that's something we've talked about a little in Topic 1 where we need to give bit of human attention to the data to check that it's good. But there were also some structural checks, as we saw a little in the last topic, that you can make to ensure that your database is consistent and reliable. We may have poor application logic. Our application that stands on top of our database may be flawed in some way and try to create problematic entries. We may have database operations that fail. It may be either that there's an

error in one of the data altering operations that we're trying to do, there's something wrong and errors thrown and that has unpredicted behavior in our database, or it might be that there's a system failure, the power goes off or some update that's being carried out on the same server causes the whole thing to fall over or there's a network error. So some of the operations in a stream of database interactions get damaged and so the database winds up damaged as a result, and of course, finally, there's the possibility that there's a user who through accidental or deliberate action causes damage to the database, causes inconsistency or incorrectness in the database. So with data coming in, as I said, there's a certain amount that can be handled with manual checks. But as we'll be seeing, automated integrity checking is built into the structure of the database and the structure of the data modeling that you've done. That the more you can tell the database system about what you expect data to look like, the more easily you can let the database system police it.
Play video starting at :2:38 and follow transcript2:38
That does not get rid of the human factor. You will always at some point benefit from having some way of checking things manually. But that's always more expensive than an automated system and the larger the data store that comes in, the larger the flow of data, the harder and harder it is to justify the cost, the effort, the time to get a human in the loop or it just becomes physically impossible for human or humans to process your data in the amount of time available.
Play video starting at :3:16 and follow transcript3:16
It's used with the application logic, can be mitigated against by a process called normalization, which I touched on in the last topic. But we'll be spending quite a bit of time early on in this topic with. Failed operations usually cause the biggest problem if they're not just a single problem. If you're just saying please just change nurse to nose, that's an operational, if it falls over, if you know that it's failed, you can run it again. If on the other hand, I've said, right, I need to do this operation and then I need to do this operation, and then this one will make sense. Then if there's a long chain of operations that have to happen for the whole thing to make sense, we need some way of keeping track of that. So saving the state, being able to roll back to a saved state is useful, but essential as well, is this concept of a block of operations or a transaction and that's something we'll be coming back to a little later in this topic. Finally, we get to this issue of user actions and damage caused one way or another by a user's interaction with the system, whether that user is an automated program or an individual like you or me, and that can be controlled just as in other parts of operating systems or computer security by a certain amount of control of user privileges, user access, user security and we'll be talking about that at the end of this topic.

So we've talked about normalization a little bit in slightly intuitive terms and over this video in the next video, we're going to into slightly more technical language and be slightly more precise about what we're talking about. But let's go back to that film status taste set, we met a rather cut down version of it, I'm going to introduce a slightly bigger version now with more columns. Here, we have some more of the many more columns in the original data, we have now title year and location as before, but we've added distributor, director and actor1, so other columns that are present in the data set, producer, actor2 actor3. So looking at this data set, there are a few things that are striking, we've already commented how quite a lot of the data recurs across

multiple rows. Any film that takes place in multiple locations will have multiple rows in the table and each case, you will get the same title, year, distributor, director and actor.
Play video starting at :1:28 and follow transcript1:28
So this is a large amount of repetition that poses a risk of anomaly if any of these go slightly wrong, we've also pointed out that the same location does not always appear in the same way in this table. But even if it did, we have repetition, so the same place can happen multiple times, this is an example of the anomaly in action. But even if this were the same string for each one of these places, it will be not ideal because we've got repetition.
Play video starting at :2:1 and follow transcript2:01
So designing a database rather than a single table dump of data, we would expect this to be represented in a slightly different way.
Play video starting at :2:10 and follow transcript2:10
We've also got the slightly strange situation that the same distributor also appears multiple times in the data set for different films because obviously, production companies and distributors appear multiple times. Here, Twentieth Century Fox Film Corporation and Twentieth Century-Fox the same company changed its name a few times in its history. I'm not entirely convinced that these are the contemporary names for the time when the film was released, but they were real names of the corporation, but it's one company, and so again, the same entity appears multiple times. We've got another schema distortion going on here that is quite interesting from a modeling point of view, which is that a single table is very poor at modeling a situation where you don't know how many entities you're going to want to list. Remember that each column has to be atomic, so we can't have complex compound fields, like a raise in any of these fields. That's why we have actor1, actor2, actor3, If you wanted to include an actor4 you wouldn't be able to. Similarly, the Towering Inferno was an unusual film in its history, was one of the first to have a co-production and have multiple distributors. Because of the nature of the modeling that was done for this data set, it's impossible for them to list both distributors, so only one Twentieth Century Fox here is listed.
Play video starting at :3:53 and follow transcript3:53
Another slightly strange aspect of this data set is, I don't know the film Smile Again, Jenny Lee, but it looks like it never received a distributor, it did not get distributed through any standard cinema release. And what that means is that the distributor field here is null, the implication of that for example, is if you thought that should be part of a primary key, we've got a problem.
Play video starting at :4:23 and follow transcript4:23
So, before we start talking about trying to normalize this from a theoretical point of view, I'd like to step back a little and just try to model the data that this is based on. Because I think what you'll find is that normalization is quite an intuitive process for most things if you model the real world situation with care, and in many ways, normalization is a formalization of that modeling and a way of checking that your modeling has gone as far as it needs to. So, how would I approach the initial modeling of this set of data? Well, I might talk about various different things, you can have a movie, a movie seems to be a thing that this data set is talking about a lot. We've also got a location, again, that's central to what this data set does, so you can imagine having these both as entities in the entity relationship diagram.
Play video starting at :5:28 and follow transcript5:28

When we talk about things recurring in different rows, often what we're talking about is things, things that have an existence that's independent of some of the other things. So, an actor does not get created just for a film and then discarded at the end of the film, much as some companies might like to try that, so, a person is an entity in this system. Similarly, when we were discussing Twentieth Century Fox, whatever name it has repeating throughout the table, what that means is that we have an entity here that we expect to model, and in that case that's the company.
Play video starting at :6:13 and follow transcript6:13
So these the first entities that leap out at me when I look at that table and maybe other ones leap out at you and then we start thinking about the relationships between them. So for example, a movie is filmed in a location or a company acts as a distributor for a movie, the movie might have a director who will be a person, it may have an actor who will be a person.
Play video starting at :6:45 and follow transcript6:45
One of the nice things about an entity relationship diagram is the ability to model the the number of entities we're going to have in any relationship, so let's have a look at that. A movie maybe in multiple locations, a location may appear in multiple movies, so that is a m to n relationship immediately and many to many relationship and as we've discussed earlier, that implies we're going to need extra tables.
Play video starting at :7:16 and follow transcript7:16
The movie to company to distributor relationship is slightly more complicated, if what we're modeling is the table we've been provided, then a movie will only have one distributor.
Play video starting at :7:32 and follow transcript7:32
If on the other hand, we were going to want to enrich that database from other sources, we would quickly find that that's also a many to many relationship. So again, we find that modeling data is not just about representing the real world, but understanding enough about our use case to be able to choose an appropriate level of modeling and an appropriate set of constraints. So I've left this as a question mark, because unless we know why we're doing this, we can't know the correct answer for that particular constraint.
Play video starting at :8:12 and follow transcript8:12
What about the person? Well, an actor, three actors in this table, many actors may appear in a movie, so an actor can appear in as many movies as they like. A movie may have up to three people in this table and again, if we're just modeling this table, that's three, on the other hand, if we're modeling the more general case, then there can be as many actors in a film as budgeted house. And the same is to directors, you may have multiple directors involved in a film in the larger case, in this particular form of data, there is a constraint that means that only one will be involved, so depending on your data sources and your use, you may have one to many or many to many.
Play video starting at :9:14 and follow transcript9:14
Now, this gives us a hint of what our tables would look like, our normalized and tables at the end of the process, but let's now return a bit to the theory.
Play video starting at :9:27 and follow transcript9:27
So the way you break things down in normalization is something called non-loss decomposition.
Play video starting at :9:33 and follow transcript9:33

Now, non-loss decomposition is breaking a relation down into multiple tables in such a way that we can always reconstruct the relation by joining all the little ones. So we mustn't lose data in the process, we mustn't lose the ability to put things back together, and part of the rules of normalization are to check that we're not losing information as we break things up into smaller tables.
Play video starting at :10:6 and follow transcript10:06
And a central concept in normalization is the idea of the functional dependency, now, a functional dependency is really something that can be worked out based on something else. So if we think about a dictionary, given the head word in a dictionary, I can always look up the definition, so if I wanted to look up Orange, I can pick up the dictionary and find the definition of orange. So the definition is functionally dependent on that heading and that means that I don't need to define the word orange every time I use it because, effectively that dictionary can always be imported into any of my prose because it's straightforwardly reducible or it would be if English didn't use the same word for multiple meanings.
Play video starting at :11:4 and follow transcript11:04
So the formal definition, an attribute Y is said to be functionally dependent on another attribute, let's call it X, if in the relation we're talking about, for every legal value of X, there's exactly one associated value of Y. So given X, we can always work out exactly what Y should be, and that gives us the idea that Y is functionally dependent on X. And let's try applying that, so we have this idea that a movie has a relationship actor with a person, so person acted in the movie. So, let's have a look, we have a movie and actor1, so Ant-Man has his actor1 Michael Douglas, so let's see which way around the functional dependency works in this case. Let's try this way around, is it safe to say that actor one will always be Michael Douglas for Ant-man in this relation? And the answer is yes, the first build actor for the movie Ant- Man is Michael Douglas in all cases, question is can we invert this? Given an actor, say, this is actor1 in a movie, that means it's Ant-Man, can we go from saying Michael Douglas was the first actor to saying therefore the movie is Ant-Man? And clearly that doesn't work, so that tells us that the functional dependency works with one particular direction, Michael Douglas as actor1 is functionally dependent on Ant-Man, but Ant-Man the film is not functionally dependent on Michael Douglas being actor1. So hopefully that's a relatively intuitive way of thinking about functional dependency and you've now got an example of how to think it through, but let's return to the table. Now, one of the things that concerns me, if we're going to be making movies and entity and give them their own table, we need to decide the primary key. And this is a slight question again, returning to the question of what the scope of our modeling should be, within the data given in this spreadsheet, title alone is unique, uniquely identifies the film.
Play video starting at :14:2 and follow transcript14:02
That's extremely unlikely in a larger case, if we were ever to add more data or update, we couldn't rely on it.
Play video starting at :14:10 and follow transcript14:10
Title and year is usually quite reliable for specifying the film, but it's not 100% reliable especially when you open up to a larger diversity of distributors, not just American ones, but what combination is reliable isn't entirely clear to me. So for example, title and distributor or a title year and distributor might seem quite a good idea, but the fact that some films don't have distributors means that that's very dangerous. I've mentioned in a previous video that it's good

to have primary key or unique integrity constraints where you know that in your model, something is guaranteed unique, some combination of fields, because it helps you police your data. However, a false constraint that, you know isn't reliably true in your model can also be a problem, and in this case, I'm not convinced that that you can reliably guarantee any set of fields is 100% safe. And so I would be strongly tempted if I were representing this to introduce a film ID, that guarantees uniqueness for any film.
Play video starting at :15:30 and follow transcript15:30
And then we can use that auto incremental, perhaps identify in the database when we need to know that the film is unique and then we don't have to place any artificial constraints on the data itself.
Play video starting at :15:49 and follow transcript15:49
If we do that, we can start to count calculate what the functional dependencies are in this system.
Play video starting at :15:56 and follow transcript15:56
So, as I said, there is one film ID for any movie and that means that title is functionally dependent on film ID, if you know the film ID, you necessarily know the title, the same for year, the same for distributor, the same for director, the same for actors and so on.
Play video starting at :16:19 and follow transcript16:19
So we have one large set of functional dependencies and then we have some other dependencies, so for example a company ID plus year might tell us the company's name. So if Fox changed their name, if we have an ID for that company then we can model the name changes without having to have multiple companies.
Play video starting at :16:44 and follow transcript16:44
Person ID will give us person name, and again if the person changes their name then we can always have a person ID Plus year giving us person name. Given a film ID and an actor number, we could say we have a person ID, so instead of having Actor1, actor2, actor3 as field names, we could now say, a film ID, the first build actor1 actor number is whoever. So these are different ways of modeling and in the next video we'll start talking about what to do with these functional dependencies and how to let them help you to normalize and so make more consistent and reliable your database design.

So far, we've talked about normalization quite an informal way just as a way of avoiding errors by breaking down a table into smaller, simpler tables that we can always reconstruct the big table from.
Play video starting at ::28 and follow transcript0:28
But in a way that reduces the redundancy of data and logic, so that information isn't repeated, isn't maintained in different parts of the database and so it's less likely to get out of sync or get wrong or into an inconsistent state. In this video and the reading that comes afterwards, we're going to be looking at some of the formalization around these concepts and particularly around the normal forms.
Play video starting at ::58 and follow transcript0:58
So there's a procedure for normalization that means that you start with the table as it comes and go through checking it against a progressively tighter set of requirements. And each of those

sets of requirements is called a normal form. You start with the first and then go to the second and so on.
Play video starting at :1:24 and follow transcript1:24
So this video will introduce the normal forms.
Play video starting at :1:29 and follow transcript1:29
The first, normal form is the simplest one to implement and to understand, and essentially it's just that the table must be a relation. So we've talked about what that means already. There's one thing I need to make a little bit clearer, more explicit and that's the data in a table must be what's called scalar, which is to say it mustn't be a complex value, it has to be atomic, it has to be a single value. So a simple example if we take the movie locations database from before, if I wanted to have one row for the film ant man, then all the locations might be an array in the location field. That's not a relation anymore because some of the structure of the database is now in the data and not in the database structure itself. And so the way to make it a relation and the way to make this legal first normal form is to split it up into the multiple rows that we've seen already.
Play video starting at :2:36 and follow transcript2:36
So this is now in the first normal form. The normalization proper, the breaking up the table itself starts with the second normal form. We've talked a little bit about non lost decomposition, and the way in which breaking tables up is possible, relies on something called Heath's there. Heath's theorem says that if a relation has three attributes or sets of attributes that cover the entire relation, we know there's a functional dependency that B is dependent on A. So for any value of A, we know what B is going to to be, then we can take B out and we get left with the relationship, the relation without B and C, and a separate table, separate relation of A and B. So if we've got effectively we make her look up table, if we know that given an A we can always know B, we make a separate table for looking that up and leave all the rest in the first table. And that's the process we use for normalization for breaking down a table into smaller tables. So the second normal form, which is where we start doing that says firstly our table has to be in 1NF, to start with, the first number form and anything that's not a key has to be irreducibly dependent on the primary key, so there's no sub structures. Let's have another look at our table, is it in 2NF? Is there any relationship that is not dependent on the primary key? Well, what is the primary key? Well, it can't be any single one of these fields and of these attributes because none of them is by itself unique. The smallest kind of the key that we can have, the smallest key that could be a primary key is title and location. So given a title of the movie and where it was filmed, we can uniquely specify in this table and and you wrote. So we've got a primary key of title and location. But distributor, for example, is not dependent on title and location, it's dependent on just title. Similarly, distributor address has a functional dependency on title.
Play video starting at :5:14 and follow transcript5:14
So we need to do some decomposition and as I said before, we effectively make a look up for those ones that have that dependency.
Play video starting at :5:23 and follow transcript5:23
So now we have the distributor and distributor address in a separate table. And for the larger table that we saw before, all the information about the movie would now be in this table, all that's left behind that's dependent on the primary key is the title and location is the key itself.

That now put our database and table into the second normal form. We have two tables, both of them in the second normal form.
Play video starting at :5:59 and follow transcript5:59
There is however, another step, the second normal form. And to introduce that, I'm going to need to introduce another formal concept and that is Transitive dependency.
Play video starting at :6:11 and follow transcript6:11
Say we have functional dependence is so we have three attributes or sexual attributes n our relation A, B and C. The name is never imaginative. We find that B is dependent on A, C is dependent on B, and because of that C is dependent on A.
Play video starting at :6:33 and follow transcript6:33
Now this is fine because we've said that we have to be dependent on a primary key and B is clearly dependent on primary key A, C is dependent on a primary key A as well, but C's relationship to A is transitive. The only reason we have a functional dependency from A to C is because B acts of the intermediate and we'll see that in the example. But this is called a transitive dependency that the functional dependency A to C occurs because B is dependent on A, C is dependent on B.
Play video starting at :7:15 and follow transcript7:15
So we want to get rid of that. That's extra structure, extra repetition and information that we can remove. So the third normal form requires the table ready to be in second normal form but also that anything that's not a key has to be directly dependent on the primary key, that is not transitively. So let's go back to this table.
Play video starting at :7:41 and follow transcript7:41
The distributor, every time walt Disney Studios comes up in this table, the distributor address is going to be Burbank California, which means we're going to get repetition again. This is an indirect dependency. There is a functional dependency from ant man to Burbank California, but that's not a logical one. It's not that Ant-Man has something about being in Burbank, it's just that and that has some relationship to Walt Disney Studios and Walt Disney Studios is in Burbank. So this functional, this is a transitive dependency and therefore to get into second normal form, we need to split this information out and now we have a look up table for distributors, for companies of where their addresses are.
Play video starting at :8:30 and follow transcript8:30
And this again makes sense, from a modeling point of view, the distributor has an independent existence, it feels more comfortable to structure it like this. And now again, we have simpler tables each with much less repetition.
Play video starting at :8:49 and follow transcript8:49
Now, these three normal forms are the central ones, they're the ones you really really must check for. There are a couple of other ones we're going to go through them. They're useful, it's much less common than a table that you've designed will preach her if it is already in the first three normal forms. And one particular example that is Boyce-Codd normal form.
Play video starting at :9:18 and follow transcript9:18
Now the chord of Boyce-Codd defined the first three normal forms, and after he published the 3rd normal form he realized that it wasn't quite as tight as he wanted to make it. The difference between the third normal form and the boys called called normal form is so slight that most examples of data that is in the normal former not Boyce-Codd are really quite odd. So there will

be something you're reading but I'm not going to give any in this video because I find it a slightly contrived situation. This would have been the third normal form if you thought of it before the third normal form was published, and it couldn't be the fourth normal form because the fourth normal form has already been published as well by the time he got round declaring it. So people talk about this is the 3.5 the normal form. The specification is that all non trivial functional dependence is depend on a super key. There will be more information about this in your reading. I think it's probably best for us to keep going and not worry too much about this one. The fourth normal form is actually much more useful. And the 4th normal form needs a discussion of another concept called multi valued dependencies.

Play video starting at :10:46 and follow transcript10:46

Up until this point, we've been talking about functional dependencies when there's a functional dependency that doesn't sit with a primary key that usually implies there's a sub structure in the table that we want to remove and put somewhere else.

Play video starting at :11:3 and follow transcript11:03

But sometimes there are structures that aren't expressed in a functional dependency and one of those is the multi valued dependency. Given a set of attributes in a relation, we can say that one has a multi valued dependency on another if for a given value of one of them of A, the thing that has, the multi valued dependency has a set of values. So whereas in a functional dependency is always one value, predictably said ant man will always be distributed by Disney Studios. What if, say in the case of towering Inferno, it's distributed by two Production companies. Every time it comes, there are two entries by the first normal form, we can't put them in the same cell so they have to be separate rows but there is still the dependency of structure that we can't deal with through functional dependencies alone. And the 4th normal form requires that the table is already in third normal form and that for every multi value dependency that these dependent on A and we show that with this double headed arrow, A is a candidate key. Simple example of that, again these examples get a bit more contrived but this is much more real life than some of these other ones, so It is worth checking for the 4th normal form. Say we wanted to know what books our students needed to read, what is on their reading list for any given student at any given time. Well that depends on the modules there on, so we can have a table of student ID module and book. Now, there aren't any functional dependence in this table, and so it gets whisked straight through 2nd normal form, 3rd normal form. But there is structure. We have repetition here. Two students who are both on the advanced databases course have this condom bag database systems, the text, they also have date's database designer as a text. So the reason there's no functional dependency here is you can't go given the value of state module, you can't know what the value of the book is.

Play video starting at :13:49 and follow transcript13:49

So that's not a functional dependency, but what it is is a multi value dependency. The book column is entirely dependent on the module column. It's just that it has multiple values for every single value in the modern economy. So, and again, this is an intuitive thing to do. It feels like the right way to structure it. We take the module and the textbook out from the column.

Play video starting at :14:20 and follow transcript14:20

And now we have two tables that look a lot simpler. It's essentially a list of the modules that the students are in and the books group module.

Play video starting at :14:31 and follow transcript14:31

And again, it feels right, if you're designing something from an entity relationship program, often you'll do the right thing intuitively, when you're doing a modeling and to some extent normalization is just a way of checking that. But the 4th normal form is a useful way of checking for some of the things that aren't captured by the 1st 3.5 normal forms. There's also 5th normal form which I'm not going to talk about in this video, but is in your reading. And again, it's a somewhat rarer, slightly more contrived an example. It's worth checking all your tables, all your designs against all the normal forms, but the most important ones to check against to my mind at least are 1, 2, 3 Boyce-Codd and 4.
Play video starting at :15:26 and follow transcript15:26
And the thing I want to emphasize here is that all of this is not just arbitrary. It demonstrably reduces the risk of anomalies by reducing repetition and also by representing some of the structures and relationships that are in our data explicitly in the database so the database itself can

We've been asking in this topic, what could possibly go wrong? And with the question of normalization that was looking at ways in which the database has become inconsistent. Because repetitions in the structure of the database meant that the same piece of information was recorded in different places and changing it in one place could result in an inconsistent state with respect to another place.
Play video starting at ::40 and follow transcript0:40
So that solves a certain sort of structural problem.
Play video starting at ::45 and follow transcript0:45
Another problem can occur not with repeated data but with multiple interactions with the database, that only makes sense as a group of interactions.
Play video starting at ::59 and follow transcript0:59
So let's take an example. Imagine our database application is a banking application, admittedly a simple one. What might be the stages to transfer 100 pounds from A's account to B's account? And this time this error is not a function of dependency. Firstly, we might want to check that these accounts exist in our system so that we've got the right account number. So we check that for A and then we check that for B. We then want to check that it's okay to remove 100 pounds. So we check the credit limit for A to see if it can sustain 100 pounds leaving the account. Assuming that has passed, we then subtract 100 pounds from A's balance, and if that's successful, we add 100 pounds to B's balance and we're done. Now, those five stages all have to be successful for this to be a successful transaction and successful transfer of funds for A to B, what could possibly go wrong? Well, what would happen if for example the final just before it gets to the final stage there's a power cut and the system goes down when we turn it back on. Well, what's happened? It might be that A now has 100 pounds less in his bank account but B has no credit has no extra 100 pounds in It. The whole system has lost 100 pounds somewhere in the process. The issue here is that the set of operations only makes sense as a block. Any one of these going wrong invalidates the whole thing and let's put this in a slightly slightly more rigorous pseudo code. I don't have really space on the slide to do all the operations at once to give us the overview but I'm going to try and tease it out a bit more clearly. So first step, we check the account number. Second step, we do the same for B. Third step, we check the credit is sufficient. Step four, we check we subtract from the balance and we subtract from the credit

as well. Because it's important that it affects A's ability to remove money as well. And we do the same, we credit B in step six, and we give B more credit in step seven. And again, any one of these breaking can be a problem. There's another problem though. They say there are two transfers that are happening at the same time, different asynchronous processes triggering a transfer of money from A to B of 100 pounds and a transfer of money from A to C of 900 pounds .. Rightm, let's also Say that a only has credit to transfer 900 pounds. Now, if the first three of these operations on each side are done first because of the way the database into leaves the operations, there'll be two credit checks. They'll check the the accounts okay for 100 pounds, they'll check the accounts okay for 900 pounds. Both of those tests will be passed and then a 1,000 pounds will be withdrawn once from the left hand transaction wants from The right hand transaction. So 100 pounds and then £00 pounds will take a beyond its credit limit even though the credit checks have been made, because 900 and 100 separately would be fine. On the other hand, If the first four steps of the right hand operation, set of operations, have been done first, the A would have the credit check and the balance withdrawal before the credit check on the left. And so the left hand set of operations would fail because it does not would not anymore have credit for that 100 pounds have gone and a credit would have gone down. Of course, that only works if we get stage five in the right hand list, if we only get stage four, the credit hasn't been changed yet. On the other hand, if we do the same thing starting with the left hand set of operations and we withdraw 100 pounds from that, then the right hand set of operations will fail. Because having withdrawn 100 pounds a no longer has credit suitable for the nine 100 pounds withdrawal. So the order in which these transactions are inter leaved, the order in which these operations are done is actually quite significant for the end result. If for example, instead of transferring from A to C, the right hand transaction was transferring from C to A. The order of operations might be even more important for whether a pulse is their credit checks. So a set of operations that would be perfectly legitimate could be rejected. How do we avoid this? The formalization of ways of trying to avoid issues that arise because of the a group of operations that should happen as a block to make any sense is formalized in a set of properties called the acid properties. The acid properties stand for four properties at immensity, consistency, isolation and durability. It's a nice catchy name, but we need to say what these things mean at ethnicity. Is that a set of operations that only makes sense as a group should be treated as a group. In other words, that set of operations should either be all performed or none should be performed. There shouldn't be a case where halfway through there's a power cut and as a result, our databases and is in an inconsistent state. So atomicity says that wants something once the database is available to any other operation. Either this particular group of operations should have been done or they should have not been done. There's no intermediate, there's no in between the consistency regulation is that the data databases never in an inconsistent state as a result of a group of operations being processed. Now, that sounds a bit like a contradiction. We can't do all the operations in a group of operations. If we can't get part way through them, we have to do them one at a time. But this these intermediate states of the database should not be available to anywhere else where they might cause damage. So if we're in the middle of changing the balance, the credit rating of an account, we shouldn't be allowing anything else to be fiddling with that because we're going to get some inconsistency. So consistency is about only exposing consistent states of the database to other operations. While we're doing this group of operations, we restrict what other things can see or do to ensure that the database

always maintains the appearance of consistency. Isolation specifies that if we've got a couple of sets of operations that all work on the same data, they can only be performed one at a time. The only way they can be inter leaved is if they really make no difference to each other. The final criterion is durability. And that's a little bit different to the others. What durability requires is that when there's a failure, any of these completed operations that we have reason to believe should be in the database will be in the database. What that means in practice is that since volatile memory, since RAM is generally much faster than non volatile memory, hard drives or solid state drives, a lot of database operations are carried out in memory. And what that means is that if there's a power cutter, if there's a problem, if something crashes, we will lose data. The durability requirement requires writing out to physical media at regular intervals in a way that's understandable and guaranteeable. And specifically, the expectation is that with if we know that a group of operations has happened, we should be able to rely on being able to to retrieve the results of that. Because they've been recorded to physical media and that's one of the most complicated of the requirements because it requires us to know when the snapshots. Well that requires the system to keep track of when snapshots have happened and what transactions have happened after the last snapshot before a system failure. And but with these four requirements, atomicity, consistency, isolation, and durability are ones that the relational database can be proven through certain mechanisms. Two be able to provide and it's one that will be looking at when we look at other models for where if anywhere there are trade offs

If we're going to be able to make the guarantees that the acid properties require, we're going to need to treat a group of operations that have to be carried out on block in a specific way. The mechanism for that is called transactions. Let's take a database. This is the standard symbol in diagrams for database. It has a particular consistent state. We have three operations that we want to perform that we regard as being a single transaction where any intermediate state could be an inconsistent state as we do this operation. We form Operation 1, we get into an intermediate state, form Operation 2 again, we're in the middle, and then Operation 3 we've finalized it. That's the version that we're happy with. Now, this could be that money transfer example where Operation 1 is checking the balance, Operation 2 is removing money from A's account, and Operation 3 adding the money to B's account. If anything happens in the middle, we've got money moving from place to place, we've got code credit checks and so on, that are all going to be for different states of the database. Really what we're saying is this middle set of states is the danger, the risk set. Their the one's that what encode inconsistent, perhaps or incorrect states of the database. It's the outer ones, the first to the last state they're the ones that are guarantee to be consistent. If we restrict access to these intermediate states, then we can start addressing the acid properties. In particular, we can start off with isolation. If, when a block of operations is happening, we restrict access to the data that might be affected by that set of operations, then we can achieve isolation. That's one of our latest star. The way to do that is just simply to restrict or lock access to the database or to particular parts of the database in those intermediate states. Now, we've privileged this initial state to the last state because they're ones we know are reliable, so one extra ingredient we need is the roll back, the ability to go back to that dependable, consistent state at any point if something goes wrong. That gives us our atomicity. Our atomicity remember, was the requirement either everything happens in a block of operations or nothing. If we have the ability to roll back whenever anything fails, and

that takes us to the state immediately before the beginning of that block of operations, that gives us atomicity. We've got A, and we've got I,
Play video starting at :3:37 and follow transcript3:37
let's have another look at these states. We have a consistent state; two problem states, one consistent state again. We need to save that state, otherwise we can't roll back to it. We need some way of recording that state at the beginning and we need some way of recording the state at the end. If we have that, we have durability. The initial and the final states are recorded in a reliable way. I put a little asterisk there because I will come back to that a little bit. The obvious way of achieving that will be to write the states at those points to persistent storage, to a hard drive. That's not reliably always the way that a Database Management System will achieve this. Consistency comes as a result of us only giving access to those intermediate states in the database, to other operations if we know that they won't be affected in some way by this known inconsistency, this known contingent state of the database. The initial state is fine, anything can do anything. The final state is fine anything can do anything. But once the transaction is happening, was the group of operations is happening, we need some restrictions. We need to be able to restrict access in an unknown, unpredictable way, and that gives us some consistency.
Play video starting at :5:15 and follow transcript5:15
The mechanism by which these reinforced by database management system is called a transaction. From the user's point of view, from the developer's point of view, this is actually quite easy. Once you've decided that a set of operations is a block that shouldn't be split, that must be atomic and that must be isolated, then you simply declare the transaction, begin transaction. Then any operations you put in will be treated as intermediate states in the transaction and then at the end you say commit and that declares the final state. You can see the effect of the Start transaction and commit or saving procedures. Things that say the now we're in a consistent state and anything in between is not. One way in which a database management system may effectively implement a non-transaction system is effectively by making every command each time in a transaction. A commit at the end of every command. Alternatively, what if something goes wrong? We can use a rollback command. If we get an error or some other system fails, we can explicitly say roll back here and the original state where you've put stop transaction of the database will be retrieved. That's all you really need to know for the vast majority of situations. A transaction can be run with just the syntax, it's a little bit more in most systems that allow you to give hints about how things will behave. In practice, there are some things you need to bear in mind. I said I wouldn't tend to split up the two languages of SQL, the one that manipulates data and the one that changes the tables and database structure. But here is a point where we have to pay attention. Because if we change the structure of a table in the middle of a transaction, many database management systems will not be able to roll that back.
Play video starting at :7:46 and follow transcript7:46
You have to check the manual, we have to check what is possible for when you're doing transactions that actually alter data table structures. That's quite a rare thing to happen, but it's important to be aware of that. More commoner a situation is a database, is a high-performing system and most high-performing systems seek to do as many operations in memory as possible because of the speed gains. What that means is that saved checkpoints, that database

is complete state to disk on not necessarily as frequent as transaction begins and commits. If you think about it, a system that's running multiple transactions once may never reach a point where there is a straightforward checkpoint, where everything is finished, all the transactions, and we can easily say the consumer consistent state. Though checkpoints are saved, there is quite a complicated dance of recording intermediate processes and what's cued up. So that if a system fails in between checkpoints, the state of all transactions can be checked because they won't necessarily have been saved to disk. Although we can guarantee with these more complex systems, the consistent state would have been saved. We can't guarantee that seems what we lost and durability has to be achieved through a slightly more complicated names. The other thing that I should mention is that table locking is not absolute. When we say we will restrict access to these intermediate state database, we have to be a bit more precise about what that means. We're going to look at some examples next, but I'm going relatively low detail. There is more detail in your reading and I encourage anyone interested to do some further reading about this. It's particularly important for high-performing database situations where there are lot of rights happening. In most of the slopes of web application type situations, you might say, until you get to e-commerce, you'll be doing a lot reads and they're less likely to damage each other in a fast concurrent database system. I'm going to give just one example of a thing that can go wrong when you have interleaved transactions. That's the inconsistent analysis. This is possibly the simplest example. There are a few others in the reading. Now, the inconsistent analysis looks like it should be quite a simple and straightforward case. Specifically, one of the transactions in this case is only reading data, the other one is writing. What's going to happen is that they're working on the same data and they select queries that come from one of the transactions are going to query different states of the data. An inconsistent result comes from the analysis. Let's take an example. It's based on the one we had before of £100 or whatever other currency unit getting transferred from A to B. There's only two steps in this version. We have a transaction start and then we subtract 100 from the balance for A and we add 100 to the balance for B, so just two update statements. The second transaction we're going to try and do is again just two statements with stock transaction and a commit on rent. Those are to check the balance of A and to check the balance of B. Say we wanted to see that the amount of money in the system was always the same because we're only doing internal transactions, that we might just check all the balances and then add them up and see what the result is. This seems a pretty harmless situation, but let's have a look what happens. We start transaction 1 with a begin transaction and at this point, let's say account A has £300, account B has £200. The total amount of money in the system is £500. We are in a consistent database statements, correct, nothing has been done yet. Transaction 2 begins, nothing has changed just issues a begin transaction command. Transaction 2 then does a balance check. It checks A's balance and it gets the answer 300. Now this is good. We know we've got a consistent state of the database, it's looking at a consistent state of the database. This is a good value we notice. We know that it's one that represents a correct state. Then transaction 1 is interleaved. Transaction 1 removes £100 from A's balance. That gives us a situation where things are being inconsistent. We have £200 in A's balance, £200 in B's balance, there's only £400 in the system at the moment. We can tell intuitively that it's really important that transaction 1 gets its other operation in immediately. Let's do that. Transaction 1 adds £100 to B's balance and that is a successful operation. We narrow back to a consistent state for the database and we can relax.

A has £200, B has £300, there's £500 in the system as a whole. Now, transaction 2 performs its second operation. It checks the balance of B and it gets a balance of 300. Again, this represents a correct state of the database. The database is consistent to the point we're making three but we have now in the process of transaction 2 concluded that A has £300 and B has £300 resulting in a total balance of £600 pounds which is not a consistent state. It's not even a state that this database has ever been. The inconsistent state, the ecosystem analysis has come about because we've had a look at two different consistent states when an analysis requires that we only look at one. How do we avoid this? Well, the textbook thing is that we need a locking system. What that means is that when transaction 1 tries to change data because transaction 2 has already said that it's selecting that data, the permission to get a write lock is denied because the transaction is ready engaged with reading. You can't write to the data when it's already being read without risk of inconsistent analysis. A locking system allows us to control who sees the data. So read access as many systems can have read locks on the same table as they like, but if any transaction is reading, then no other transaction should be able to write to that data. Similarly, if a system is writing, nothing else should have any access, it shouldn't write, it shouldn't read. There is another option that some database management systems will do, which is if you know that a transaction is only going to be reading, you can make a temporary copy of the last known good state at the beginning of the transaction, and then you know that you're only giving read access to it. Then you can essentially give it the old state of the database that you know is correct to do its analysis whilst the other transaction is busy rewriting things. So if you know in advance that a transaction only reads and it's efficiently efficient to make a copy of the state of the database at that point, or that table at that point, you can actually just feed it stale data, and know that that's safe.

Play video starting at :17:36 and follow transcript17:36

We've talked a little bit about how locking your database can allow interleaving of safe operations. If operations aren't going to effect the same data, if they're not going to result in inconsistent analysis, then we can permit operations to happen. How can we tell? To make a database fast we need multiple concurrent operations, we need as much not to be locked as possible at any time.

Play video starting at :18:11 and follow transcript18:11

To enable that, we need to be as precise as possible about what's safe. One thing that we can say is that if we can guarantee that the result of performing interleaved transactions is the same as if a some ordering of those transactions, one at a time, then that's safe, that's an allowable, consistent process because we know that that means that there's isolation. If two transactions running at the same time produces the same answer as two transactions running one after another, then they must be to some extent isolated. They must be durable. Serialisability; if we can be sure that that's available, if we can be sure that our interleaved set of operations gives us a serialisable result, then that gives us the guarantee that our transactions are safe. How can we be sure of serialisability? That gets a bit harder, and we can say that if all the operations operate on completely different data, then we can interleave in any way we like, and it'll be safe. If all the operations are SELECTs, we're also safe. Once we start getting rights then the analysis is a bit more complicated, there's a negotiation of locks and various other rules, and again, if you're interested, there's more detail in the reading.

Play video starting at :19:50 and follow transcript19:50

Really from a user point of view, the only thing you need to know for many cases about ensuring the ACID properties are preserved is you get an ACID preserving database management system, and you declare your transactions. Just as with keys and with normalization, you give the structure to the database and then the database take care of managing your structures. The same is true with transactions. If you have a block of operations that will produce inconsistencies in the logic of the system, or that only make sense if they get carried out at once, or which only make sense if they're all carried out on the same state of the database, then you declare them a transaction, and by doing so, you give enough information to the database management system that it can manage, making sure that it works and manage the asset properties. It does that using locking strategies, and it does it using failure recovery strategies, under the hood. But from your point of view, the ACID properties will be preserved if that's what the database management system is capable of, and if you give it enough information about your operations to be able to manage it.

In our explorations of what can possibly go wrong in this topic, we have tended to conclude that if we make our structures and our logic explicit, if we structure our data in a way that reduces the burden on the system to carry multiple pieces of information to maintain consistency, then it becomes much easier for human or system errors to be handled appropriately or avoided entirely.
Play video starting at ::45 and follow transcript0:45
But some human errors or deliberate human actions can still cause trouble.
Play video starting at ::53 and follow transcript0:53
And in this final set of activities we're going to be looking at that the possibility for malicious or accidental damage caused by users.
Play video starting at :1:6 and follow transcript1:06
Now, there are always possibilities of deliberate malicious action and this xkcd, I've reproduced just one pane of is an illustration of what's called a sequel injection where a a data value is provided that usually contains a quotation mark, that's going to close the quotation marks around whatever it's been put in. So, if someone puts this into a sequel statement of my school statement, for example, and it has a quotation mark in, it closes the string we're trying to put, in this case a student's name into, and then the attacker has the opportunity to insert their own sequel and executed on your system. And that itself can be mitigated, there are software based ways of avoiding sequel injection, but to some extent, to a large extent, one of the things that's surprising when these sequel injection attacks are successful is what it says is that some system that should only be able to insert a student's name into a data table is being given permission to do a hell of a lot more.
Play video starting at :2:42 and follow transcript2:42
And so, there are other ways than security hardening on the level that people normally talk about when they talk about SQL injection for controlling what a user can do. And that's really what we're going to be looking at in the coming videos and activity. So, what can go wrong? Simple injection can put malicious code into a normal operation to make it a malicious operation. And there are ways of working around it. But again as I say, the damage should be limited by controlling what the user is able to do. Malicious agents getting direct access to the database and again getting access to the database at a high enough level of security to do

damage. User errors, so a user who's only intending to do one thing does other that's more damaging. And of course, another issue that only becomes an issue here is incorrect sharing of data. That's not anything to do with an error in the database but it's giving access to the database in a way that you don't want to. So, what a lot of these have in common is that they come down ultimately, to use a control. Although, sequel injection is a more come across in a future topic, something that you can and should deal with on a software level, or fundamentally often it comes from also poor user control. So, we'll be looking in the coming video at controlling user power and user privileges of acronym.

In this video, we're going to be looking at how you can control the access that a user or a script has to a SQL database. We had that example in the previous video of maliciously inserted code having the ability to drop tables, to remove a particular table from the database when someone thought that all they were doing was inserting a new student into that table. There are ways that we'll get to in future topics of cleaning the input data so that it has no risk of suddenly starting new commands. But one has to ask the question, why was it that someone who was only putting data into the database, why did they have permission to remove the table? Why did they have permission to restructure the database when all that they needed was privileges for putting data in? If you can be very precise about what's expected at any given time, what commands, then you can protect the system against the wrong commands being run. Users in SQL have quite fine-grained controls available. It's somewhat more precise than, for example, a file system or operating system's controls. Users have permissions for the ability to create or edit other users. They have privileges for creating or editing databases. They have privileges on tables on particular bits of data. We have multiple granularities with which we can control what the users can do.
Play video starting at :2:12 and follow transcript2:12
It's important when you're defining your database and working on what you're gonna do with it, to consider the question of what users you're going to have and what's the minimum set of privileges you can grant them are. Because if you always only permit them a narrow set of privileges, if you can give a user only the right to select data, then the risks to your database are much reduced. Sometimes your user might need separate roles. In other words, it might be that a user has effectively multiple ways of logging in or that they login to an application that has different logins at different times, so different access points in order to ensure that they have only the permissions for you expect them to do at a certain point in time.
Play video starting at :3:10 and follow transcript3:10
The way we give permissions to a user is quite simple, even though there's a rich set of privileges we can give, the command is quite simple. That's effectively grant the ability to do stuff on the bits of the database we want them to do the stuff on to the user or users. Just grant keyword, on keyword, and to. We want to grant privileges on structure to user. An example of that, we might want a read-only account. We're saying, we want to grant select privileges, the ability to use the select command, on planets and moons to this user.
Play video starting at :4:6 and follow transcript4:06

We have grant, which is followed by the set of things that you're allowed to do. In this case, just the select command, on an analyst of structures, and those can be databases as a whole, and then to the username of the user.
Play video starting at :4:31 and follow transcript4:31
We've done grant select, if you wanted to give more permissions, we could for example say, grant, select, insert. In the case of the Bobbi tables from the previous video, if the school administrator had insert privileges but not any other privileges, then drop tables would never have happened. Grant, select, insert if you're only expecting people to add data to a table and not to restructure the table, then this is a useful combination of commands and privileges. Grant, select, Insert, drop allows you to remove the table. It would be an unusual combination of privileges but it's possible, we can combine these arbitrarily. Of course, this goes hand in hand with actually creating the user. The user's created with simple commandments, create user, name of user, identified by, and then the password. There are other ways of doing this setting of passwords, and some of them are system dependent. But bear in mind that typing your password in plain text and sending it to a user's not necessarily the best way to do it. What I've set up here is a simple example, is one that will expire as soon as the user logs in. So we've got a slightly insecure system, we've given them a password that will communicate to them, and then as soon as they log in, they have to reset it themselves. The encryption of the password internally within the database is managed by the database management system. I should say that when you look at the MySQL database on your lab systems, if you go show databases, you'll see there is a system database, there is a table for users because the requirement of a relational database system, is that everything is a table. Even the management of users is through relations is through tables. Let's return to the statement. There are other things we can give through this, through the ground command. We can add a little clause at the end. So having done a grant on and to clauses, if we say with grant option, it means that this user can give other users up to and including the same privileges that they have. By default they can't pass their privileges onto others, but with grant option allows them to do that. We can also revoke any set of privileges just with the command revoke instead of grant, and it has exactly the same syntax except that instead of saying, we ground to, we revoke from vocal on planets, from username will work. That is really the basics of user specification and granting them privileges. You'll see the list of privileges in the documentation. I'm sure you can work out how that will fit into an application system rather than user-based system. One other thing that is worth mentioning is roles. Roles have been a part of the SQL standard for a long time, but they took a little while to emerge in implementation. MySQL has only had the roles for a relatively short period of time. The idea here is that if you're an astronomer, you might have the ability to add, say, planets and moons to our list. Let's imagine that's what astronomers spend their time doing. We create a role for astronomer. Then when we have individuals, we can assign them all the privileges of that role. So we grant, insert and select on planets and moons to astronomer, there's no user called astronomer, this is just a role, a generic set of privileges that we can hand around. We can say grant being an astronomer, grant astronomer to username. This slide is a reminder if I've forgotten to put capitals for the to, and that won't face the SQL interpreter. Commands are case insensitive. They didn't mind whether their capital or lowercase. We tend by convention to use capitals for all SQL commands because it's easier to read. Not least because most of our table and field names are going to be mostly lowercase or camel-case.

Roles are a useful way of assigning a set of privileges to several users, and managing users who come into and out of different roles over time. So if a member of staff becomes a manager, we give the manager privileges and then we can revoke them if it's a temporary promotion or whatever. We can handle these sorts of changing sets of privileges much more easily with the role system.
Play video starting at :10:21 and follow transcript10:21
Just to conclude, the real point of this is to minimize the privileges of a user and then in the process to reduce the impact of any mistakes. Because although they can still make mistakes, they can only make mistakes in the areas we'd expect them to have write access. By constraining their privileges, sure, we reduce malicious ability to change tables or to access data that they shouldn't have access to that maybe confidential. But we also reduce the risk of their errors being very damaging.
Play video starting at :11:5 and follow transcript11:05
We can and will give uses too many privileges when you're playing around with the databases in the next topic, you'll almost certainly not create restricted users, you'll be playing around as a root user of the database. In a production system, it's really worth reducing and reducing the use of privileges as much as possible. That's an aspect of the database application design rather than an afterthought in the part of a sort of security policy afterward.

In our discussion of what could possibly go wrong, how do we make our databases reliable, we've considered a selection of different sources of problems. It can be bad data that comes in, It can be bad logic in our own application, it can be software or hardware failures or it can be malicious or incompetent users. All of these are ways in which trouble can happen, that we can't stop, we can't prevent any of these from happening without huge investments of time and money and effort and redundancy, and even then there's always a problem. So what we try to achieve and particularly with relational database systems is to mitigate against those errors to protect and harden the database to minimize those errors into reduce the effects of them. So for bad data coming in we have our integrity constraints and we also have normalization to reduce the risk of bad data in one part of the database, making the logic make no sense. Similarly with poor application logic, the normalization process should make things better. When things fail, the transactions and snapshots are particularly important. And finally malicious or incompetent user activity can to some extent be mitigated by good user policy.
Play video starting at :1:53 and follow transcript1:53
As I say, we can't prevent mistakes, we can't prevent errors, we can't prevent hardware failures or software failures, but we can mitigate somewhat against their effects in ways that are predictable.
Play video starting at :2:9 and follow transcript2:09
So that is the end of the database reliability topic. And in the next topic we're going to be trying out the database is a bit more. We're going to be looking at a bit of real data and we'll see what we can do with our access to the database from other languages and other platforms. After that in future topics will be coming back to relation database theory to look at making the databases more efficient and other more advanced database topics.

We've now talked a lot about how to structure a database, how to make sure that it's reliable, but we haven't really done very much with it. We haven't talked about how you talk to a database, except in sort of basic syntactic terms. In practice, you will be doing that mostly through libraries in other software. And we're going to be looking particularly at some of the more common ones. And hopefully you'll see the common factors that will then allow you to adapt to whatever your needs are. We're going to look at node, so JavaScript, we're going to look a little bit of PHP, we're going to have a go with Python as well. And with these basics, you should be able to work with pretty much any language, any environment where you need to talk to a database. And we'll look at the commonalities as we go. And as we walk through it, we'll explore some of the things you need to bear in mind when you are addressing a database, through user provided data and through your own code.

In this topic, we are going to be working a bit more intensely with my secret database in labs. I'm going to revise some of the commands we've already met and fill in a bit with a few other ones. These will be the main commands you're going to need in all your SQL and MySQL interactions. The first one is very much a reminder is SELECT. SELECT gets information in the form of a table as always from one or more other tables and the form we've seen in SELECT a list of columns or star, and then FROM, and then the list of tables and then any constraints we want to put on the results [inaudible] into a WHERE clause. The three main clauses you need to remember are SELECT, FROM and WHERE. Now you can do approximate matching. We've seen in the WHERE clause using equals or less than or greater than. You can use a LIKE keyword to retrieve text with wildcards. In this example where name like '%er' that percent is acting as a wildcard and any number of letters, any so and going between. Another advantage of LIKE, is that unlike equals, it is case insensitive. In this case the er could be capitals, or they could be lowercase letters.
Play video starting at :1:56 and follow transcript1:56
The second command is for changing data that's already in the database and that is UPDATE. UPDATE takes a table, UPDATE Planets. What we say is we want you to change certain values of certain columns for rows that match certain criteria and so we have the UPDATE telling us what we're doing, SET telling what to change and WHERE saying the criteria. UPDATE Planets, SET name equals Mars where name is equals Mers. Here it is correcting a typo that we've already got.
Play video starting at :2:34 and follow transcript2:34
UPDATE is relatively simple command so we can move straight on to the next one which is INSERT. INSERT is for adding new rows where UPDATE was for changing existing values, INSERT UPDATES as I add new ones. Again, INSERT takes the table, INSERT into Planets and then optionally we can name columns. If we don't put any names, it'll assume we're doing all of the columns and it will give an error if we didn't give enough information to fill those columns. In this case we're specifying INSERT into Planets (name, diameter) and then VALUES introduces the sets of information that we're adding, so in this case name is Gethen, diameter 9483 and that goes in brackets. The reason it goes in brackets is we can add as many new values we want just by separating those bracketed groups with commas. We can have as many new values being inserted just by adding these and it's very useful programmatically when you're generating stuff from another script, rather than having lots and lots of insertions to have

one insert command that does multiple [inaudible] or ones. We can also insert the results of a query into a table. INSERT into Planets name, SELECT DISTINCT name FROM Moons would run the subquery, the sub select SELECT DISTINCT name FROM Moons, find all the planet names in there, and then in INSERT them into Planets. That can be quite useful, especially when we're normalizing or manipulating tables, that we can transform information from one table and put it into another. We've done SELECT grab information, INSERT to add new rows, UPDATE to change existing values. Of course the next one will be DELETE. DELETE again deletes from a table just as INSERT inserts into a table, UPDATE changes results in a table, so it immediately takes the tables input. and then a criteria, or criteria. DELETE FROM Planets where diameter is less than 2,500 will remove Pluto from our table. Those are the main operations that change data in a table. Now we get to the ones that are more about changing the database structure itself. We have DROP, for instance, which removes the table DROP TABLE, deletes the table that we name, so DROP TABLE Planets would delete it. Doesn't just get rid of data. It deletes the whole table. Less dramatic is TRUNCATE. TRUNCATE TABLE removes all the entries in it, but leaves the structure and that's a bit different to DELETE FROM Planets where star, because DELETE triggers all the cascades in foreign keys and deletes related items. TRUNCATE is just a straightforward removal of data with none of the logic attached.
Play video starting at :6: and follow transcript6:00
CREATE we've already met. CREATE has a lot of different forms, but for the point of view of some immediately useful things, CREATE database we've met very briefly. It's a very simple command. CREATE TABLE is the one we will run more often and that we've met a lot of time so I don't need to talk too much about it. CREATE TABLE, name of the table, and then in brackets, the list of all the columns that we're going to want in that table and then key information and constraints and other information about the table. We'll be playing with that much more in this topic. The final one I wanted to mention is ALTER. If you create a table and something isn't quite right about it, you don't need to delete it and start again. You can use ALTER TABLE command, just to tweak something, say to change the type of a column or the text encoding used, or the names of a column or something like that. ALTER TABLE, name of the table and then the thing you want to change in this case, add column, diameter, integer, where say you had it as a decimal before. Those are the main commands you use within a SQL environment itself. Now it's a little bit more I want to introduce before we get into the labs and that is getting data into and out of the database. Firstly, if you want to export some data as a CSV file, as a text file. You can do that by adding a little bit extra onto your SELECT, so SELECT star FROM Planets, gets all the columns from Planets into OUTFILE and then the path name, the name of the file and that would just create a file in your file system and ready to be imported elsewhere, will be used elsewhere. If you've got a CSV file that you want to import, then you can use LOAD DATA INFILE which is a command we've not met yet, and LOAD DATA INFILE , name or the file name into table and the name of the table you want to import it into. We're going to be using this in the lab. I'll be showing you in another video so you'll get a chance to see that before you use it yourself.

Later in this topic, you're going to be working with JavaScript and with Python to access and explore databases. But I thought it'd be a good idea to get a bit of practice with MySQL from the

command line before we go into those. We've got one lab coming up, which we'll be doing just that and I've launched into a similar environment, in fact, I slightly cheated. You'll be loading some tables in from external data. I've already done that. I've just launched MySQL and I'm going to say, "Use Movies", which is the new database that we'll have created. "Show Tables", the usual thing.
Play video starting at :1:5 and follow transcript1:05
I'll make this a little bit bigger for now, we might need to shrink it in a moment. There are three tables, there are actors, there are composers, and there are locations. Locations is the table we've already met, the San Francisco data. If I were to do "Describe Locations",
Play video starting at :1:31 and follow transcript1:31
you can see the fields. We've got the title, the year, the location, some random information about it, the producers, the distributors, the director, the writer and the top three billed actors. If I go, "Select Star From Locations Limit One Starting at Zero".
Play video starting at :1:59 and follow transcript1:59
We've got a film called 180, which is in the table at the beginning because it's a number based one. We've got two other tables that you'll be using. They are both data that I've downloaded from Wikidata, which is an online community maintained resource that I just searched to get this information from. We'll be meeting it a little bit later on in the module. But those tables are composers, let's get a description of composers,
Play video starting at :2:45 and follow transcript2:45
and you can see what I've done is film the titles, year, and then the composer and their gender and date of birth. I've done that just as a blanket thing over Wikidata. Some of those films would have been filmed in San Francisco and so you'll be able to work out who composed the films that are located with those locations in San Francisco. You may be interested in that. The other table I made was actors, so again, "Describe Actors", and here we've got what I've called ENName, which is the English label put in Wikidata. The date of birth and then name in native language or name in own country, which is a separate field in Wikidata and isn't present in all cases and then gender. Just some very basic information and from that, we're going to try and connect these databases. This isn't a normal foreign key relationship because the actors and composers tables come from a different data set to the locations data. But still, we should be able to do something. Let's have, "Select Title, Year,
Play video starting at :4:23 and follow transcript4:23
Name and Date of Birth,
Play video starting at :4:35 and follow transcript4:35
From Locations Join Actors
Play video starting at :4:51 and follow transcript4:51
on Actor 1 Equals ENName." Let's not mess around with approximate matching, let's do it precisely.
Play video starting at :5:6 and follow transcript5:06
I will just put a limit on that and see the first 10.
Play video starting at :5:18 and follow transcript5:18
Here we go, we've got repetition there, let's get rid of that repetition. "Select Distinct," that's because some of these have multiple locations and if they have multiple locations, it will appear multiple times in the database. Here we go. Here's a set of alphabetically the first, however,

many titles, and now we've got the date of birth of the first actor. Because I put name, rather ENName, I've got some slightly weird things, let me fix that as well. Let's have the "English" name. Now, yeah, we've got a set of dates of birth, year's titles. Now, if we want to explore this dataset, we probably want to be able to summarize it because we've got quite a lot of rows. If you want to deduce anything about dates of birth or about gender, we're going to want to be able to summarize the data, and for that we need some commands that we haven't learned yet.
Play video starting at :6:26 and follow transcript6:26
Grouping data, this uses a part of the single standard called aggregate functions, and the idea is pretty simple and it's quite easy to use, but it's quite powerful as well. Let's say I'm going shopping, and I'm not just going shopping for myself, but also for my mother, and for my neighbor, so I've got this list from the receipt from the products that I bought. It's got things that I bought for myself, things I bought for my mother, things I bought for my neighbor. I want to be able to bill appropriately say you only have much. Naturally, the first thing I would do is launch a database and type it all into the database, and if I do that, what I want is firstly to identify the products that have been bought for the different people. Then having identified them, I want to group them together so that all the ones that I bought for my mother, are in one place, the ones that work for my neighbor in another place, one for me in another place, and then having grouped them, I want to summarize them, and in this case, by adding up the price. The end product I want is the price of the, I didn't care about the price of individual items. I care about the price of items when I grouped or aggregated those rows, based on who they were bought for. Actually, from quite a large table, all that I want in the end is three rows, and the price for each of the bought for and data points. How might that working in a civil environment? Well, so the simple thing you might expect if we want to find all the ones that all the items that were bought for mum, we could just do the query select start from shopping where four equals mum, and then we'd have to somehow add up the results. But the point is, there's one field. We just need to focus on the boat for, and there's a special constructor for grouping things by one of those fields, and that is the group by. After we've done our table constraint, after we've done that when constraint, we can say group by bought four and then all the elements that have the same thing in that column will be bound together as a something. Ready to be reported on using aggregate functions, and that's whether something here is. We can do select boatful because in a group by statement, each one of those rows has all of its members with the same value for bought for, either be or neighbor or me. When we get to a field like price, there'll be lots of different values, so we can't just report it. We have to apply, and aggregate function on it. Say, how to summarize a clump of aggregated data. In this case, the summary we want is some. If we could select bought for some price, and then using the grouping, it will come together, the prices, and then it will apply some and it gives the total that we want. Some isn't the only aggregate function, but it's a good one to illustrate how it works they work pretty much in the same way. We have some which we met. We have average standard deviation, and variance functions, there are few other standard statistical functions available. We have Max and Min, now where some and average work pretty much exclusively on numbers, Max and Min will also work on other data types and behave slightly differently to this. For example, the particularly useful for dates, Max, will be the most recent data, Min or the one that's further into the future. Min will be the one that's further back in the past on the timeline. Maximum to be quite useful from that point of view. Count is completely agnostic to what the data is, it just counts the number of things that we have

aggregated in our group. The number of items that I bought for my mother, for my neighbor, and so on. There's also accounts, distinct function, and no distinct gifts in the brackets before the name of the thing you want to count. That simply removes duplicates in the list. If I wanted to know how many different products I'd bought, and I had two [inaudible] of milk that I bought from my mother, and they appeared separately in the table, then count instinct would match those together and count those as one..

Play video starting at :11:58 and follow transcript11:58

Then for strings for text, we have group_concat, which simply takes all the different strings, all the different text that's being grouped together and clumps it all into one text optionally with a separator. Those are the aggregate functions and now let's use them.

Play video starting at :12:20 and follow transcript12:20

Let's use some aggregates. Select count. Let's say this is the title. No. Let's SELECT Gender, COUNT(*), FROM Locations, JOIN Actors on Actor1 equals ENName and now we need a group, group by Gender.

Play video starting at :13:10 and follow transcript13:10

What's this going to give us? This is giving us the number of film location combinations where the first build actor is male and how many where it's female and there don't seem to be any transgender first doctors in the States. What we see is we've got 1,227 for male and 390 for female. Now, there are several things that make this slightly strange, for example, the fact that we're not normalized, so these lists are of movie location combinations. Film that has multiple locations in San Francisco will be more heavily weighted in this case. Actors that feature in those will be more common.

Play video starting at :14:6 and follow transcript14:06

Still, it's quite interesting to try switching that to the second build actor. The second build actor in this case, is more commonly female. You can see there are 692 females, 629 of the ones we can identify and again, there's all the distortions from the location information. But we get some quick surveys. Let's have a look at the third build actor.

Play video starting at :14:40 and follow transcript14:40

This time it's overwhelmingly male, again. Not scientific and there are though, within the dataset that you've been provided, there are ways that you would be able to make some of that bias go away so we can get a more accurate measure if, for example, we normalize the database or if we simply do a sub select that only chooses a unique film titles and actors. But still this is interesting start. We can also see how slightly strange this dataset is if we do something like SELECT Actor1 COUNT(*) FROM Locations. Or which organized group by, Group by Actor1 ordered by. I'm going to have to give the counter name, so let's call it as appearances ordered by appearances and I want that disc descending and I want to limit it.

Play video starting at :16:7 and follow transcript16:07

Here what I am saying is I just want to know how many times each actor features in this database and I want to order them so that the ones who appear most are at the top. I'm going to limit that. I'm only going to look at the top 20 actors. This is an indicator of how weird this dataset is. The number of appearances in this particular data table, the San Francisco location's table, is not necessarily indicative generally of how many times a particular actor appears as first listed actor in film, Jonathan Groff, Hugh Laurie, Clint Eastwood aren't necessarily the people you'd immediately think of as appearing in that. But of course, Clint Eastwood has been

in films that play very heavily in San Francisco and in the Bay Area and same for all the other ones in this list. It's quite fun to explore and try to understand the data you've got. This has been a brief introduction and please have a go at it and then talk about it in the forums about what you found from the dataset, do feel free to try, normalizing this dataset, make new tables and just see what you can find out?

For the rest of this topic we're going to be connecting to the database not from the command line but from various different programming languages.
Play video starting at ::21 and follow transcript0:21
Most of our interactions with a database aren't through the command line tools provided by the database management software creator, but through libraries. And so we're going to be having a look at how those libraries work just as a quick tour of the various different languages, the different approaches, and most importantly what they have in common. Because there's quite a lot for we're going to be looking exclusively at MySQL, but for all the different database platforms there's a lot in common between all the libraries, so we're going to do a quick survey and visit the different libraries. Now some of the elements that they all have in common. They will create a connection that will almost always be a persistent one. They'll send commands, and there'll be a mechanism for doing that. And then the results will come in some form that we can then act upon and display or analyze the results. So let's take those one at a time. Creating a connection, and we'll see this in various different platforms but it will be one of two different shapes of things. Either we'll set up a connection, we'll define it by saying information like the host URL, the username, the password, if there's a database we want to start using then we can provide that, and then having set up an object that's a connection object we then run a connect command and off we go. The other sort of general paradigm for doing it is to do both of those steps in one, and you'll see things like connect host, username, password, database as a function that does the creation of the connection as an idea and making the connection at the same time. And we'll see that, for example, in PHP where actually this connect function doesn't necessarily create a new connection, it looks to see if one exists, if one exists it returns it, if one doesn't exist it creates it. So that's creating a new connection. The next thing is we want to send a command, we want to make a query, we want to insert something, or create a table. It's almost always string based, so you'll generally make a query string and then something like an execute command or a run query. And that may give you results, as an array for example, or it may give you some sort of resource object that you then run fetch me the data as a function on. And the reason for this distinction is the library has a choice, it's provided by the SQL server, the database server, either to grab all the data in one go and hold it locally, or to retrieve it from the server a little bit at a time and let the server buffer it.
Play video starting at :3:50 and follow transcript3:50
And from the point of view of resources for the server generally it's quicker if the server can get rid of the whole query as quickly as possible, so if the client grabs all the data in one go the server can then go and do other things.
Play video starting at :4:10 and follow transcript4:10
The second form where you grab it all is a bit more resource intensive for the client but usually not that much. But the libraries give a choice, often, between these two. So we're executing queries, we've learned how to do the connection, and then the other thing that will happen is

we'll get this response. And usually the response is something a bit like an array one way or another, it's something we can iterate over in a loop. And within that each row of data, the theme we're iterating over, could be an array and we'll address it numerically, or it could be an object and we address it by the names of the columns. The interaction with the server always provides the names of the columns, so it's possible for the language, for the library to make it an associative array, an object that we can retrieve from, and allow us to retrieve things by the names of the fields. But alternatively they may just give us indexes, give us an array. So broadly speaking, those three areas are the ones we're going to be looking at and seeing how they work in the different technologies. We're going to be looking at connecting, we're going to be looking at sending commands, and we're going to be looking at using the results that come back.

So why might we want to use JavaScript to talk to our database? Well, probably the most common reason would be because we want to run a web application with a database back end.
Play video starting at ::23 and follow transcript0:23
So our first lab for this is going to be doing just that. We're going to look at using Node.js, Express and then Mustache as a way of building a quick web application that allows us to talk to our database and use it to create web pages.
Play video starting at ::45 and follow transcript0:45
I suspect that a lot of you will have used these tools before. They're quite common and that's why I've chosen them. But I wanted to give a quick introduction before we get into it. Firstly, the way the lab set is up, [COUGH] we're using the NPM package manager so that you can install all the packages without too much effort. So what will have happened already, there will be a directory for your app in the lab. There's already a package.json file that says what your app depends on and so it lists the dependencies. And then if you from the command line, from the terminal run npm install, it will then pull the libraries that it needs to run the lab.
Play video starting at :1:33 and follow transcript1:33
Now, any import lines which you'll see in your JavaScript will be pulling libraries that have already been loaded by node.
Play video starting at :1:43 and follow transcript1:43
So there's a package,json file ready in the lab and it just lists which libraries are needed. It's a quick easy way of organizing it. And we've got MySQL which is the library that allows us to interact with the database. We've got Express, which is a web server. We've got Mustache-Express which is a templating language for that web server. So the express server, let's start there. You may have met this before. I just thought I'd talk through some of that. There's a lot of code already there in the lab that you'll be using and so I want to make sure you can understand it and get to it quite quickly. So we require express, we import it. And as I said, that library will already be loaded if you've run npm install and then we create an app. That's what the sort of standard terminology for it. We declare a web ports which is the port which we're going to be mounting in. In a normal web application, you might be running on 80 which is the standard HTTP port. Quite commonly though, that's used by Apache or Nginx. And then your JavaScript or other web servers will be running on other ports within the system on local host. And then you'll just connect those two. And so Apache will be all that's seen as a web server by the outside world. And your express server will be internal. App.get is a way of

declaring a particular path as being one that you'll listen for HTTP requests on. So in this case, the slash means top level of this. So local host colon, 8808 slash whatever we're going to be listening on. And then you give it a call back. You give it a function which I've abbreviated because I ran out of space. It would be a function with two inputs, the request object and the response. So the request object is what the client has sent to you and the response is what you're going to do to them. So you will at some point do some stuff in that get and then you'll do response.render which will send something to the client. And then you just tell it to listen at a port and it will do that and it'll listen for those ports that are declared in the get.
Play video starting at :4:20 and follow transcript4:20
So those structures in the lab, you should now be able to recognize. The next thing is Mustache. Now Mustache is a templating language. It's called Mustache because the people who named it thought those curly brackets looked a bit like a mustache.
Play video starting at :4:36 and follow transcript4:36
It's built with a lot of curly brackets hence the name. And so say there was a variable called elephant length. You can write normal HTML and then just embed a couple of curly brackets and the name of the variable elephant length in. So what this would do, this the elephant is elephant length meters long. Code would replace the {{elephantLength}} with the actual value of that variable within JavaScript. And so you'd end up with something like the elephant is three meters long or whatever and that's really convenient. You write your HTML template with a load of these named variables in it and then you render the template, feeding it a load of data. What you'll see in the example you've been given is a slightly more complex case but much more common for database queries, which is a loop. So usually the database will return you an object which is essentially a table. It's a list of rows and each row will have a lot of named columns. So say let's stick to the elephants for now.
Play video starting at :5:49 and follow transcript5:49
Say we've got a bunch of data called elephants.
Play video starting at :5:53 and follow transcript5:53
This bracket bracket hash construct is a loop construct. It says for every single one of the things in elephants, which in this case we're going to say is a set of rows from a database. do the middle block. So between the hash and the slash two lines below is the range of loop. Just do this until you've run out of rows.
Play video starting at :6:23 and follow transcript6:23
And then within that, if each of the entries in that iterable object is an object, so it'll have a name, it will have a length in this case. So you can use the standard curly brackets syntax for all the elements in the loop. So this is the way that you'll see it in the material that you have in your lab. It won't be about elephants unfortunately. But this is the most common way of dealing with a load of results from a database.
Play video starting at :7:5 and follow transcript7:05
The third component is the MySQL database itself. How do we connect?
Play video starting at :7:15 and follow transcript7:15
We connect using, first we load the data, the database library const mysql or whatever you want to call it = require('mysql'). So that's loading the library. And again we can do that because we used npm install to install that library already.
Play video starting at :7:36 and follow transcript7:36

Then we run createConnection on that object, a method of that object. And we give it a host, a user password, database or whatever.
Play video starting at :7:47 and follow transcript7:47
And the result then is a connection.
Play video starting at :7:51 and follow transcript7:51
It won't run the connect yet. So we have to run a connect command on that declared connection and then it will make the connection. And when it does that, I've given you connect without an argument here. You'll see in the lab, I give it an argument because if something goes wrong, you can give it a call back, you can give it a function that it will run on error That will allow you to report an error. And that you'll see in the lab.
Play video starting at :8:21 and follow transcript8:21
So we've connected, now we want to query. Querying is as simple as running a method of the database called query, which is nice and simple. So you need to give it the query string itself, select something, update something, insert something whatever. So it just looks like a standard MySQL command.
Play video starting at :8:47 and follow transcript8:47
And then because JavaScript is an asynchronous language, a lot of the later languages that we play with, you'll run the query and then when the answer comes, you'll have more lines of code underneath. In a JavaScript environment like this, what happens is you give it a call back, you give it a function that it should run on completion. And when that result comes, it'll run that function. In this case for a database query, the function has three arguments. The error, so if there's an error, it will give you an error message. Otherwise it'll be blank. The results which will be an iterable object with objects for each row with the field names and it will give you the field names. I haven't used the third argument in this example, but you do get the field names as a sequence as well. And you can do whatever you like in that function. When you have a look at it in the lab, what we're doing is in that function, we're going to send an HTTP response. Because it's only when we get a result from the database that we're ready to display some HTML we can actually start doing some things. There are a lot of other lines in the lab. It's not a particularly big block of code but it's quite fiddly, which is why I wanted to talk through the major sort of milestones and major sort of stopping points.
Play video starting at :10:33 and follow transcript10:33
Try to look at all the different bits, even the bits you don't have to change and see if you can understand what's going on at all the parts. If you can't do, go onto the forums, do ask your tutors.

You've now connected to a SQL database from a node based web server, retrieved data from it, and displayed it in a web page. We're going to go on to looking at doing something similar, getting a query from a database using the Python MySQL libraries. But before we do that, I'd like to talk a little bit about another wave templating technology, and that's PHP. This is a quite a popular language for building websites. However, it's not one that we're currently able to support within the lab environments. I thought I would describe it and then explain how it works. It's not too far separated from the way of the node and the Python libraries work. We'll just talk a little bit about how it does work. As before, we will divide that into the way to connect, the way to

query and the way to handle results. The Libraries called MySQLi, not least because it took over from the library that they had, but previously that was good MySQL, and they couldn't name it the same name. You start a connection with Just initializing the object, new MySQLi, open brackets, host name, username, password, database. This is the version of that connection. That is a single step one. As soon as you do this, it looks for an existing connection. If one exists then it gives you that, and if it doesn't have a persistent connection to that database, it starts a new one. Executing MySQL commands on that connection is as simple as providing a string with the query text in it to a method of that object code query. Just like with the others that will then get executed. If this is a select query or something else that we expect to return a value, then we can look at the results, the thing that it returns, and that's an object of the form the MySQLi result which we can then process. What can we do with it? Well, firstly, as with a lot of the other cases, we can treat it as if it were a list itself of some sort, it's retirable. We can do a for each on it and we can access each row that we get out values of that row by column name. Like in this example. We can also grab all the results in one go. Then we can treat it as a two-dimensional array. We can say what is the six rows, second column. By default, the factual method fetches the results as a plane array, which is why I'm using numerical indexes for here. But you can also provide it with an option Virtual MySQLi assoc, which allows you then to read the data as by the name of the table. There's different ways provided by the MySQLi interface to work with a database. Broadly, this is very, very close to what we're seeing in the Node.JS example.

So far, when we've used a library to address a MySQL Server, we've done so only to select data. We've only been reading from the database, not writing. When we start writing, there are few things we should bear in mind. First thing is, thankfully, pretty much any command in SQL could be just sent through the socket through the connection and the library's connection to the server and just run. There isn't much of a difference between an update and a query.
Play video starting at ::54 and follow transcript0:54
We should bear in mind that there is a security considerations and there are ways of reducing the risk of having malicious material inserted into our commands to run other commands in the database. The other thing is there's another layer of separation from the database. There are a few things we may need to keep track of, especially if we have quite a complex relational structure with [inaudible] Making the updates, as I say, really, all you do is you make the insert command you otherwise would from your language of choice in this case, JavaScript. Then you simply run it as a query.
Play video starting at :1:50 and follow transcript1:50
If you're doing it programmatically, obviously you're going to construct it by concatenating strings with extra bits of information and so on. But essentially the principle is the same. You build the string, that's going to be the command, then you send it off to the query. But this is where the security consideration comes in. This thing called SQL injection, SQL injection, as long as those inputs to the insert come from another source. It doesn't need to be an insert, it could be even into a select. If you're using user supplied data, and by that I mean, either things have been typed into a form or that have been provided to your script by some sort of HTTP response that has formed data in it for example, you can't necessarily trust it to be clean and the example we had before, the Robert quotes Drop table students from XKCD. If we've got a quote in there,

and we use that example I showed earlier, the way of putting it together. That quote will close the thing that makes it a string and we'll drop into command writing mode. We have to be careful that when we provided data to go into one of these queries, it's clean. Now one way to reduce the risk of the drop tables problem is to control the user's privileges to make sure that the script only has the ability to do the things we want and can't drop the tables. But we can actually have a bit of hygiene about strings that are being supplied. One way to do that is to escape the things we're about to put in ourselves to make sure that quotes always have a slash before them so that they're inserted as text rather than as semantic elements. We can also restricts the possible operations in other ways. Let's take an example here of a way of cleaning the data without explicit escaping, letting the system do the escaping for us. If instead of constructing that string, the insert command manually ourselves, we use this syntax of question marks, so we insert into actors values, open bracket question mark, comma, question mark, comma. Then there's a special part of the query operator in the node MySQL library that allows you to provide data to it in the query operators. If your query string has question marks in it, then the next argument to the query command can be an array of values. Those values are automatically escaped before they're inserted into your command. That's one way of making it safer and also a bit tidier.
Play video starting at :5:12 and follow transcript5:12
Another way of ensuring that only very controlled things can happen is to use stored procedures. Now this is something that exists on the SQL database management side rather than on the library side. But if we were to write this command, you declare that the end of your command is not going to happen when you have a semicolon, because otherwise as you create the procedure, you'll pop out of the definition things. You create delimiter slashes or whatever you wanted it to be. Create procedure, give it a name, and say what is input are, in this case, name, gender, date of birth. Then you say begin, and the command, and the name, gender, name, and date of birth here, are the one's about the arguments to the adapter procedure. Then you declare end and we're done. We just make sure that the delimiter get back to being a semicolon before we do anything else. Now when we call it from our library, we can say call adapter, Richard Gere, male, 1949-08-31. You're wondering why I'm picking on Richard Gere. I think he may be missing from the actors list in the dataset, so that's why I'm using him.
Play video starting at :6:40 and follow transcript6:40
The advantage of this, is that we can simply give the web user an execute access on this stored procedure only and give them nothing else and again that protects us from accidental or malicious running of other code. Now, the stored procedures, there are ways of running those on every library, although the other method I use, the one which involved cleaning the input or letting the query command clean input, that particular form of it only exists in a MySQL library in JavaScript, all of the libraries have something similar for cleaning input that goes into a MySQL command. It's worth looking that up for the library that you are going to use.
Play video starting at :7:38 and follow transcript7:38
The only other thing I'm going to give to you, talk about to tidy up this thing of inserting things into databases from Libraries. If you're using an auto-incrementing ID rather than a primary key that comes from your data you'll want to know what the new ID for something you just inserted is. Because for example, if you're doing a relational database, you might need it as a foreign key in the next thing you're inserting. Within MySQL or a standard SQL environment, you can simply select last insert ID, and that will give you the value you want. From a library there will be other

ways of getting that last insert ID and again, that's library-specific. This example here is again one from Node Janus. You simply give the query function a callback function. The callback function gives you an error value if there's an error, a response if it's just the results, and the names of the fields. That results will have an insert ID property, so you can access that. If you run an insert command and you know you're going to need the ID that was associated with the new ID that generated for that row, then you can use this property, this trick, to make sure you got the right ID for going to the next step.

The last library we're going to look at for connecting to our MySQL database is from Python. The reasons for connecting from JavaScript or from PHP are quite simple. They're very popular for building web servers. If you want to build a web application that uses a database back-end, there's a high likelihood that you'll be using one or other of those technologies. Why would you use Python? Python is used for a lot of things. One of the reasons why it's so popular is that there are a lot of libraries already in existence that you can use to build your applications. In particular, Python is quite popular with data scientists and scientists in general. It's quite a common use case to want to get data into and out of your pattern environment and to control it from the Python environment. It's popular for graphing. There are several libraries you can use for that. We're going to be looking at a particular part of Matlibplot called Pyplot. It's popular for data analysis and machine learning. Libraries like Numpy and Pandas and Google's TensorFlow that allows deep learning. Again, things that have very heavy requirements for data. Also quite significant for certain data structuring, there are libraries for scraping the web or for wandering around a website. Something like Beautiful Soup, where you go through a database and extract structured data from the webpages and build a structure out of it and that structure will often be the database. There are lots of the common use cases for Python that might expect to have a database back-end. The library for connecting to MySQL and interactive in MySQL from Python is, as I've indicated before, quite similar to the ones we've met already. You connect using a connect function, which takes the usual set of arguments. You query by taking that connection and getting what's called a cursor. The cursor is going to be our accompanying friend for making the queries and so on. When we make the query, we will then execute the query, which will be a text string as usual, on the cursor.
Play video starting at :3:9 and follow transcript3:09
Then we'll iterate over the cursor, or we could iterate over the cursor and print some index of it. We get each row from the table that comes back and we can access that row like an array. Another alternative, like we've seen in the other environments, is the ability to fetch the whole structure once using a fetchall function.
Play video starting at :3:38 and follow transcript3:38
The other thing that was worth saying is that if we tell it that we want dictionaries, associative arrays, objects, if we want something that we can access by the name of the column rather than the index, and that's what's called a dictionary within Python, then we just need to say, we say it when we create the cursor. Then we can access the columns in a row by name. In many ways this is similar to what we saw in the PHP example earlier. Now, one thing that's a little bit different that you will have to look out for if you're updating data from Python, is that auto commit is turned off by default, by the MySQL connection. What that means is that everything

you do is regarded as being a transaction. As soon as you create a connection, it starts the transaction effectively. Then all your executes, our treated as being part of that transaction, they will not be committed to the database until you send a commit method, the commit command. When you're using Python for changing data particularly, you need to run commit at the end. There are ways to turn your auto commit on. But this is quite a useful thing anyway, as long as you bear it in mind. We're going to be using Python in an example. We're going to be graphing some data. Try that and see how it goes in the lab.

It's no coincidence that there's a good deal of similarity in the ways of interacting with the database from the different programming languages. When you install a relation database management system like MySQL or Postgres, what it does as a server is listen on a particular TCP port for certain sorts of connections and certain sorts of triggers. The libraries are sometimes called by MySQL connectors for that reason. So what they're doing is connecting to MySQL and using its existing infrastructure. And so we'd expect there to be a great deal of commonality because essentially the underlying operations are being done by the same software. And that's true whether you're connecting internally to a local host port or from an external client to the MySQL server, and that's completely transparent. What is different then is the interface that the writer of the library builds on top. And that can choose different options whether to commit automatically at the end of each operation. Or whether to treat the interaction is being a transaction. How the connection is carried out and presented to the user. And to some extent in some of the other libraries, we haven't looked at whether the structure of the query itself is translated into the programming language natively. So whether you send a complete string for a command or whether you run a query method or an insert method. But all this is to some degree, just interface changes on top of an underlying core that is MySQL. And this makes it very easy to move between the different environments, different libraries. So now we've met those, we're going to, in the next topic, be looking at ways of checking the speed and improving the efficiency of our queries.

A relational database management system can seem a bit like magic. When we make a query we don't do what we would do in say JavaScript or for say, for every member of this table, put this into an array that you are temporarily making, and then go through every member of this table, check whether it's equal to this, and then store it also in the array and carefully and in detail telling it what to build and how to build it. In practice with a relational database system, what we say is, I would like a result that behaves like this. Go produce it. We trust the system to do that in an effective, accurate, reliable, and quick way. In topic three, we looked at ways of ensuring that that was reliable, that the operations of the database were reliable, and part of that was about ensuring that we avoid mistakes by telling as much to the management system as possible so that it constructs things appropriately. It can decide an appropriate behavior. Something similar is the case for speed and efficiency. We can't tell the database how to go about its business of extracting the tables we want, and so what we need to do we need to understand what it needs to know and help it to make an efficient and effective query. In this topic, we'll be looking at ways of understanding what the database is doing when it's building a query, and in some cases, changing structures or providing structures to make that query faster.

Relational databases are fast, and for the uses that most people encounter most of the time, a database is easily fast enough, that you need never worry about efficiency or optimization. If you've got a normalized database, especially you've done a lot of the hard work in giving information about the structure of the database, to the database management system, and you can rely on it to do its job at optimization. But that's not always the case. The bigger your database gets, the more users there are requesting information, writing information. The more complex your queries, the easier it is to find something that brings everything to a standstill. It's important when you get to that, that you understand what the factors are, that they going to be contributing to that slowdown. Let's have a look at those. What's expensive in running a database? The most expensive aspects of any operation in a database, are searching, which will often involve checking values on every entry in a table, or comparing them with other entries. Sorting; sorting generically whereas searching you might look at every row, that's one per row, in rows in times of the work, sorting at its best is in log in, so it takes more than one unit of time for every row that you encounter. Sorting in particular is a very expensive task. Copying data is also expensive. Effectively it's reading and writing, and if you're doing joins, we've seen how they can multiply up the number of rows that you are going to produce. Writing that somewhere is going to take up space and time. These are the things we're going to have to look out for, when we're analyzing the sources of problems. Let's take a simple example of the searching essay. Here's a table, these are the gold medalist winners of the Olympic women's archery contests. I've got all the ones going back all the way back to 1972, accurate at the time of this video being made. If I wanted to look through all these to find a particular individual, I would expect to have to go from top to bottom and just walk through it. Given a name, I'd go through until I find the row that matches. Now, worst case, if it's the last name, it's going take me n, where n is the number of rows. On average, after all, I might find the first person. It might be the first person in the table as well. On average, we'd expect it to be n over 2. If I have a table with a million rows, I would expect it to take 500,000 reads to find it. The worst case would be a million rates. What happens if we sort the table? You might think this will make no difference, after all I still need to look at all the entries, and the name might be anywhere in there. You might think it would still be order n, order n over 2, worst case Zebedee something and they're right at the bottom. In this case though, we can use a different algorithm. We can use binary search. I'll give an example, if I'm finding Mi-Jin, she's somewhere in this table.
Play video starting at :4:29 and follow transcript4:29
We're going to use binary search to get there. I've chosen the way I'm going to do this, is it going to give me a worst case? I look in the middle entry, this is why it's called binary search. I divide the table into two, I'll look at the middle entry. That's Kim Kyung-Wook, and I say, "Well, is Yun Mi-Jin before or after?" Since we know it's after, we can throw away half of the table. Now we've got a half-size table. Again, we look at the middle point. That is not divisible straightforwardly, and as I say, I'm going to go for the worst-case, so here we go it's Park Sung-Hyun. Again, her name comes before Mi-Jins name in the alphabet, so we throw away the first half of our remaining table. We're now down to a much smaller table. Now I go to halfway through this and I get Hyang-Sun. Again, I need to choose whether to discard the first step on the second half, this time. Her name comes after missions, and so we can discard the rest of the table, the second half of the table, and that gives us our answer. It took us four steps. That took us four guys and it's a log case. The worst case is, we'll have to look into log n rows. Best case

obviously, always, is that the first thing we look at is the right answer. Best case is always one, the average case here is log n. To illustrate the difference. If I have a million rows and I'm using just a straightforward walkthrough defined. I have to look at 500,000 rows. If I have a million rows and I can use binary search, I have to look in approximately 20 rows, worst-case. That's quite a substantial reduction. When you declare a primary key on a table, this is exactly what the database will do. They will sort according to the first part of primary key, so that on the assumption that if that's what you want to search for, then it can do the binary search into very quickly. Using that, using a sorted table, what's called clustered indexing. It's as fast as the indexes we can come onto. Its really fast. It doesn't use any extra space at all. But it is actually a physical ordering of the data, and you can only choose one of those. You can only choose one column that you're interested in indexing as the primary index. But within those parameters, this is an incredibly useful, incredibly powerful way of substantially reducing search time.
Play video starting at :7:44 and follow transcript7:44
There are, however, indexes as well. We can stop worrying about the data in the table and make a separate structure that we can then search on. Usually, there will be some tree structure or a hash table. I'll talk about the B-tree and the hash table in turn. Not so much in how they're implemented, but in the effects of them. You will probably have met both of these in data structures courses already. There are also specialized indexes. Particularly common, ones for geometric or spatial data. For example, if you have mapping information and you want to find all the points that are close to this latitude and longitude, find me all the restaurants that are near me. Conventional indexes are quite bad at that. There are special indexes for those cases. There are also domain-specific indexes and feature indexes that come up, we may be mentioning in a later topic. One big advantage of an index is that you can make an index much smaller than the table that it's based on. You can keep it in memory or keep parts of it in memory. That can make it much faster, even if it's not much more efficient than, for example, doing a binary search. It may also know something. You may know something about the structure of the physical storage and before you make your index. If you know where the file blocks are, if you know how the data is laid out on disk, you can index in such a way as to optimize the file reads when they do happen.
Play video starting at :9:41 and follow transcript9:41
B-Trees and hash tables, let's start with B-trees, balanced trees. We have, a worst case and typical case that's the same as the binary search and it's going to be some sort of log of n. That's fast. I mean, for most purposes, this is incredibly fast. One really nice thing about B-trees and it's true of the binary search as well, is that because you know everything is in order, because you can look at limits of whether data is. You can do ranges. Range based search is very quickly. You can say find me. All the cases where people have balances between 200 pounds and 5,000 pounds very quickly. You can also do some approximate searching. If I say, I want to find actors whose last name begins with j, then if I've only got a single name column, this won't help. But if I wanted all actors who are called James something, then the fact that the index is sorted already means that I can jump straight in. Some sorts of approximate searching are greatly helped, others are in no way affected. Hash tables are super fast. Hash tables get rid of the problem of scaling entirely in terms of retrieval, and not necessarily in terms of the problems of actually making a hash and the space taken up by the hash. But they don't scale with the size of the data.

Play video starting at :11:29 and follow transcript11:29
In certain unusual cases, if there's a lot of clashes of the bins, if you wind up with a load of data that gets repeatedly hashed into the same code, then you can get order n, but it's quite rear. Almost all cases will be constant time. Now, we talk about the expense of an operation in terms of whether it's constant time, whether it's linear, whether it's exponential or whatever in terms of n. But databases are fast and reads simple operations are fast, so an older log in, we said on a million operations you're doing 20 reads, and it may well be that those 20 reads are faster than one hash algorithm being run. If you've got an expensive enough hash algorithm, it may be that your data needs to be that much larger before you benefit from it and you'll find that a lot of databases default to using B-trees rather than hash tables because they're fast enough, and the cost of maintaining a hash table become a little bit too great until the data becomes really quite big.
Play video starting at :13: and follow transcript13:00
Hash tables are also poor for approximation, they have no approximation, they have no range support. You will be taking something and converting into something else and hashes are almost always designed to spread things out very broadly. Things that are similar will be put to very different places in the hash table. Whereas, for a range or an approximation search, you want them close together.
Play video starting at :13:29 and follow transcript13:29
Yes, you get no range-based retrieval, particularly, unless you have particular classes and hash out with hashing algorithms. There's a concept called locality-sensitive hashing, which might put things that are similar together. But that's a very specialized thing that is not common in these sorts of hash tables and it increases the risk of clashes. Those are the two main ways of indexing.
Play video starting at :14:1 and follow transcript14:01
What are the problems that come from just to share numbers? What are issues? Let's walk through an example and see what happens. We have a banking system here. We have an account's table, which for reasons essentially of screen size, I've given three fields, the account number, the customer number, and the balance and then we have a customer table that is similarly because I wanted it to fit on half of the screen, has three columns, which has the customer number, the name, and a telephone number. For clarity of this example, I'm going to assume that no account has more than one customer set, there is no joint accounts, but that one customer can have multiple accounts and let's say, we are large bank, so we have 20 million accounts and 10 million customers recorded in these tables. Let's do some Maths and see what happens. Well let's do some queries and see what happens.
Play video starting at :15:11 and follow transcript15:11
I want to get my staff to call all the customers who have gone into overdraft. Here's the query, it's very simple. I want the account numbers, the balance, the name, and the telephone number of all those accounts where the balance is below zero, and I want to sort it by balance. There's a couple of different elements to this, and the order in which they happen may turn out to be important. Let's try a particular order. We're going to start by looking at the tables we're getting it from. We've got two tables, accounts and customers. Let's do a cross join of those to start off with and then we can fiddle with the criteria on that new copied data well copying 20 million rows from one table, ten million rows from another and combining them gives us two times 10 to

the 14 operations, and that data has to be stored somewhere. I didn't calculate the number of bits or bytes that we're going to be storing this in. It's already quite a lot.
Play video starting at :16:35 and follow transcript16:35
Now, we can look at the constraint. Cases where the balance is below zero. We need to look at every single row and check the balance. That is going to be 2 times 10^14 operations. We can look at all those and we're going to delete the ones that are below zero. Now, I didn't know what proportion of my customers have an overdraft, let's say about half. That means that they were going to be 10^14 rows left in this table after I have done the filtering. Now, I'm going to sort it by balance and that takes n log n time, say that is log of 10^14 is enough that it'll give us two more orders of magnitude. It'll take us to about 10^16 operations to do the silt.
Play video starting at :17:40 and follow transcript17:40
This is already taking us to quite a large number when we do the final check to only keep those cases where the account numbers match, so the customer numbers match in both accounts tables and the customer table, that's 10^16 operations as well. This is order nm log nm, which is the multiple of the two tables sizes times the log of the two table sizes, it's huge. This is 10^16 operations of the order of magnitude. That's clearly not ideal. It's big tables though maybe we can't get it better, but let's try. Let's do a different order. Let's start off by trying to match accounts with their customers. Here we're going to take the 20,000 accounts and we're going to match them to the customers. Now, we know that there's a search we can do that means we can do it log 10^7 to find the matching customers. Assuming either there's an index or there's binary search available to us. That takes us 2 times 10^7 times log 10^7, is about 10^8 operations. We can now run the balance query. We know how many rows there are in the result because it's going to be 20 million still, so we're just filtering 20 million results. We need to look at every single one because they're not ordered by balance.
Play video starting at :19:31 and follow transcript19:31
We don't know again, I'm going to guess that 50 percent of them are overdrawn. That's going to give us 10^7 results. We want to sort it because the last operation is sorting and that's n log n. That takes us back to about 10^8 operations. We've gone from order nm log nm to order n log n. We've gone from 10^16 operations to 10^8 operations. Which means we've knocked a 100 million times faster than we were before. So getting the right order of operations is really important. There are optimizers in the Database Management System that try to work out what the best way of executing a command is.
Play video starting at :20:32 and follow transcript20:32
Indexed and sorted tables will make this faster because they can make all three of those things that slow things down faster. They can get rid of the copying problems, searching problems, and the sorting problems. The order in which things happen has a huge, huge impact.
Play video starting at :20:52 and follow transcript20:52
The order is important, but also which indexes to use when and the creation of indexes which can be done explicitly is a part of that too. We can make those fresh indexes ourselves. But you'll notice that some query optimizers also make indexes on the fly. We'll see that later that the MySQL engine will in certain conditions keep track of the data that it's copying as it does it make an in-memory index and then use that later on in the query.
Play video starting at :21:32 and follow transcript21:32

Anything that requires copying or reading data has a tendency to increase the expense and operation. Anything you can do that reduces the data before copying will have a huge impact on the final speed.
Play video starting at :21:54 and follow transcript21:54
How do we work out when we need to optimize? The standard specifies some ways of unpicking what's going on and there is a bit of variation in how that's implemented, but we can look at how it's done in MySQL. Let's take a query that we've already met on a dataset that we already know. Select everything from movie locations and actors where first actor named movie location actor one is equal to name. We've done this before, it's not particularly slow, but it is still fairly operations intensive. Before this video, I ran it on some data and I had a look at what was involved. This is incredibly unscientific, I was doing it on a virtual machine on my laptop while it was doing other things and we'll see the results of that. But it's instructive anyway, I think. If we put the word explain before the select, we get some interesting information out of the system and it will tell us what operations is going to do on which tables in which order, and we can have a look at that and see if it's reasonable. See if it's going to be too expensive and if it looks a bit problematic, either change your query, create some indexes, change the order, or even override aspects of the query optimization process itself.
Play video starting at :23:37 and follow transcript23:37
In this case, when I ask it to explain, the first thing you would do is look at movie location's table. I've selected some of the information from what it gives, you'll see much more when you try it. It says that the select type is simple what that means is that it's not using any complicated structures like sub select or unions. These are things I haven't gone through yet in this course and one of the reasons for not doing that is that you should make sure you're very fluent in SQL before you do them, because they tend to be quite expensive. Sometimes it's very tempting to use them. But often there are other ways of writing the same thing that a more efficient, so only use these slightly more exotic structures when you really need to in general. What else does it say, possible keys now, there are no keys that are going to be useful to us for this, because actor 1 is not a key of movie locations and I suspect that actor name is part of a key in actors, but isn't a whole key, there's certainty set nothing in movie locations that's of any use. It says it's going to be looking at 1819 rows and which is not too much and filtered. What filtered means is can it reduce the data before it does any copying? If this is a 100, it means 100 percent of the data has to be looked at. If its one, only one percent of the data has to be looked at. This is looking everything that's quite expensive and extra information, it doesn't have any, there's nothing else he wants to tell us about this. The most significant information is usually in rows and filtered. We've got 18, 19 rows and 100 percent passes to the next stage. When we make a table that joins these two, it's going to be 1,819 times something. The second stage in the explained is actors, it looks at the actors table, it says the select type is simple again absolutely possible keys? No, there's nothing it's willing to use to achieve its goal. This time around is 40,000 rows 39,175, which is a bit more substantial. It can filter them though, because it will already know something about the movie location actors. So it reckons that only 10 percent of those 40,000 rows are going to be useful to it and will use where to get that relationship, and it's using what it calls a hash join. This is a relatively new thing, if you try this out with an older version of MySQL, you will not get hash joins. Hash joins has been a way of a significant speed-up in doing this query, because this is a case of building a hash table as it copies data.

This is that case, I was saying about where indexes are built by the query engine as it queries. When it copies the movie locations, it will fill a hash table and now it can use that, and that's how it can filter so effectively 10 percent of the data out. Really it's expecting to copy, take that 40,000 down to 4,000 quite easily. When it does the copying, so filtering and copying. In practice, this procedure took 60 milliseconds approximately, with all the other overheads unscientific, only done once. But it gives us an idea of scale of things. Now, I made one change. I created a new index on the movie location's table to index actors. Now, it will have an index to look at. We won't need to build any. The question is, how does that change things? Well, I ran explain again. This time around it wants to look at the actor's table first. It says, it's not going to use a key for this, it's got 39,175 rows. It's not going to be filtering them at all. It'll be looking at the where I close that. Second stage is movie location. It's doing these tables in the other order because it's got this key. Now, I can use this new index. That will mean we only needs to care about seven rows. Because it's going to be able to do this much more quickly. They filtering 100 percent no extra information. This is going to be really fast now because it's indexed. It took almost twice as long. Why did it take you almost twice as long? Probably just because my computer was doing something else at the time. This is within double the time, it's not and that extraordinary. It wasn't as big speedup as even its own analyzer thought it would be. But it wasn't slow to start with and disk access is of this order of space. It's not necessarily slowed down much. I'd have to do more checks to see that and that's something to bear in mind. Don't just use single queries to generate a timing, Iterate them lots of times to be sure. Let's persist. Let's try another aspect to this. Let's create an index on the actor's names in the actor table and do the same thing again. If we do that, then it reverts to the original order. Now, it's starting with the movie locations 18-19 rows,100 percent will be used, it's got its keys ready. It doesn't need to make new keys. Then it does the second one it's also got keys. This time 18-19 rows because it knows exactly what they need to be because it's already got the case. This is not going to take long, it's not doing any searching anymore. It's just using its indexes, when it was run, this time we've got a big difference. This is less than a 10th of the time taken. Absolute terms, this is 55 milliseconds away from the first one. The second one was 50 milliseconds away from the first one, they look similar. This this is 10 times faster.

Play video starting at :30:55 and follow transcript30:55

A bit of tweaking around with keys, working out what's going to work may be useful when things go beyond the millisecond level of searching. Or when you're going to have 100 people all hitting the disk at the same time.

Play video starting at :31:14 and follow transcript31:14

There is a science underlying indexes you can work out what's expensive and why. But for the actual costs, doing real life experiments are useful. I should explain that the explain command has an explain analyze version. If you don't explain, analyze select, it will give you the breakdown of what it's going to do, but it will also run it. It'll do an estimate of how many operations? How much cost it thinks each bit of that query will take, but it will also time it. It'll do a dummy run where it doesn't give you the data and it will time how long each stage takes. These numbers that I've given you have come from that. There's a bit of overhead for they explain, analyze, procedure but it's quite a useful way of seeing it. The interesting thing with that 110 milliseconds. The second case was that it's estimated the cost was much lower than its estimate of the cost for the first one. But as I say, I'm using a virtual machine on my computer on

a solid state disk in a system that's doing a lot of other things at once. There's quite a big variation in the amount of time things take. We've established the usefulness of indexes of appropriate query strategies and being aware of which queries are going to be more expensive and which ones less expensive. What sorts processing work best on the system. That's also something that means that in some cases, you might want to bring that to the web server or even to the client to do the last bit of processing to save your server a bit of effort. The rest of this topic, we'll play around with that a little bit more and we'll look at other strategies for speeding things up.

Up till this point, we've spoken about the normalization of a database as being the gold standard for reliable accurate databases. But sometimes reliability and speed can be in conflict.
Play video starting at ::31 and follow transcript0:31
What does normalisation bias? Normalisation can reduce disk reads by allowing us to separate out bits of data, and perhaps only read some of that data when we need it. By breaking a table up into units into components we don't need to read the whole table, all the columns all the time. And also we don't need repeatedly to read information that is repeated in the database. That's our one retrieve table has in multiple places. So it can dramatically reduce disk reads, it can reduce the disk use, it can reduce the need for integrity checks because we've removed repetition.
Play video starting at :1:26 and follow transcript1:26
It reduces storage use for the same reason that it can reduce disk reads, that it gets rid of repeated information.
Play video starting at :1:35 and follow transcript1:35
It increases the use of joints, and as we've already said, joins are potentially expensive operations. They involve some sort of searching, they may involve consulting in index. And even if that index is an order log in retrieval for millions of results that adds a couple of orders of magnitude to the speed, or to the cost of doing an operation.
Play video starting at :2:11 and follow transcript2:11
So one way to avoid that problem is to denormalize to glue our tables back together.
Play video starting at :2:20 and follow transcript2:20
Why would we do it? Because it reduces the number of joints we have to make, it reduces the complexity of queries. And effectively what it is, is instead of doing that joint select we have a cash. Now we have a version of the results of that select.
Play video starting at :2:40 and follow transcript2:40
It's important to understand what we're doing when we do that. So if we do it, it should be a deliberate thing after we've normalize the database. This is not a license not to normalize your databases. So reduces the use of joins in your operations it may result in faster databases. You're weighing up the amount of data you retrieve with the changing complexity of the queries. And that's something you'll only tell for your specific needs.
Play video starting at :3:20 and follow transcript3:20
So you can do it easily enough. Here's an example, if I wanted to create a new table that had actors joined up to their listings in the separate actors table. So we've got the movie locations table, we've got the actors table. We join them up into this special bigger table just by making that query with the joints in it. So we joined the tables together and we make a new table off

both of them, and that is denormalized now. As I say, it's only sometimes faster, but it's also risky for dynamic data. For all the reasons we've talked about in the integrity sections of this module so far, having duplicated data is dangerous. Duplicating data takes time. If you have something that gets updated frequently, you can get out of sync tables. So this is something you should do with care. There are other ways of looking at it though, we can create a view. So this is a structure within the SQL standard, a view is a saved select. So we declare this select that joins the movies with their actors as a view and we can treat that view as a table. We can select star from SFMoviesAndActors, and that will give us all these fields when we want. Obviously, in real life, what I've admitted in the previous ones, we've got three actors listed in that thing. So if you wanted that to preserve that structure, we've had up to one actor to actor three, and it would be this gigantic query. And that does get more expensive as you add extra bits. And so this would also simplify our queries, our queries would look nicer. There's a problem which is that these views are dynamic. All that happens when I do select star from SFMoviesAndActors is that it translates that into the big query. And so all the expense of the big query is preserved.
Play video starting at :5:44 and follow transcript5:44
However, the standard gives the opportunity for something else. What the SQL standard calls a snapshot. And what a lot of the implementation school things like embodied views, or materialist views. And is a saved view, and these are neat. So what this means is that if I make that query, the create view, I can save the results of it, not to a table but to view. I can treat that view as a table. I can even change update rows in it.
Play video starting at :6:20 and follow transcript6:20
But all that does is it changes the parent tables in response. I can give privileges or to that view separately to the separate tables. And when those tables, when the parent tables are updated it regenerate the view.
Play video starting at :6:40 and follow transcript6:40
And that gives us the advantages of a denormalized table with none of the risks.
Play video starting at :6:49 and follow transcript6:49
Unfortunately, it's not implemented in MySQL, there are ways of hacking it. You can use some of the programming abilities that come with my skill to build your own. But if you want this facility, if you want embodied or materialized views will need something like postgres to be able to build it. So to summarize, databases are really fast. And key-based joints are also optimized for heavily in a database, especially when you've got indexes and keys around. So it's really important to understand what it is that you need. Don't optimize until you have a speed problem, or you're aware of speed problem is coming. And when you do, do the safe optimizations first. Evaluate your speed, evaluate your efficiency, think about how big your database is going to be in the future, because that's the way the hard thing is. That if you're going to build your database and you start testing with 100 rose, but you know you're going to have 100 million, you're going to have problems evaluating and stress testing. So always think about the size and efficiency. Always think about whether you need to optimize and where you need to optimize. And try the safest approach possible before you reach for the denormalisation.